

Sistema de gestión de entradas para eventos deportivos basado en microservicios

Autora: Gutiérrez Tabango, Johanna Maritza

Tutora: Yacchirema Chicaiza, Silvana Abigail

Facultad de Ingeniería y Ciencias Aplicadas, Universidad Central del Ecuador

Carrera de Ingeniería Informática

Trabajo de titulación modalidad examen complexivo, componente práctico, previo a la obtención del Título de Ingeniera Informática

Quito, 2026

Índice de contenidos

Introducción 1

Planteamiento del problema 1

Justificación 2

Alcance 3

Objetivos 4

Objetivo general 5

Objetivos específicos 5

Teoría 7

Arquitectura de microservicios 7

Seguridad: autenticación y autorización 8

Transacciones y concurrencia 9

Diseño de APIs REST 11

Aplicaciones web de una sola página 12

Calidad y pruebas de software 13

Marco de trabajo Scrum 14

Metodología experimental 17

Metodología de desarrollo 17

Arquitectura de la solución 19

Modelo de datos 21

Reglas de negocio implementadas 24

Herramientas y tecnologías 25

Estrategia de pruebas 26

Uso de herramientas de inteligencia artificial 28

Análisis y resultados 30

Funcionalidades implementadas 30

Pruebas de base de datos 36

Pruebas funcionales de API 37

Prueba de concurrencia y carga 38

Defectos encontrados durante las pruebas 40

Discusión de resultados 42

Conclusiones 45

Recomendaciones 47

Referencias bibliográficas 49

Anexos 52

Lista de tablas

Tabla 1. Planificación de los sprints del proyecto 18

Tabla 2. Reglas de negocio y mecanismo que las garantiza 24

Tabla 3. Herramientas utilizadas 26

Tabla 4. Resultado de las pruebas automatizadas de base de datos 37

Tabla 5. Escenarios de prueba E2E en Apidog 37

Tabla 6. Defectos detectados y su corrección 40

Tabla E1. Endpoints principales por microservicio 58

Lista de figuras

Figura 1. Arquitectura de componentes del sistema 20

Figura 2. Modelo entidad-relación de las tres bases de datos 22

Figura 3. Catálogo de partidos 30

Figura 4. Mapa de asientos de un escenario 31

Figura 5. Mapa de asientos de un coliseo 32

Figura 6. Reserva con cuenta regresiva 33

Figura 7. Entradas con QR 33

Figura 8. Mis reservas 34

Figura 9. Panel administrativo 34

Figura 10. Pantalla de configuración 35

Figura 11. Reportes con indicadores 35

Figura 12. Control de acceso 36

Figura 13. Resumen de la ejecución de la prueba de concurrencia y carga en k6 39

Lista de anexos

Anexo A. Plan de sprints del proyecto 52

Anexo B. Reporte de ejecución de pruebas de API 54

Anexo C. Resumen de la prueba de concurrencia y carga 56

Anexo D. Repositorio del proyecto 57

Anexo E. Contratos de la API: endpoints principales 58

Anexo F. Manual de instalación 60

Resumen

La venta de entradas para eventos deportivos puede presentar dificultades cuando varios usuarios intentan reservar el mismo asiento al mismo tiempo. El presente trabajo presenta el diseño y desarrollo de un sistema de gestión de entradas deportivas orientado a evitar reservas duplicadas y facilitar la administración de eventos. Utilizando una arquitectura de tres microservicios: autenticación, partidos y reservas. Cada servicio dispone de su propia base de datos en PostgreSQL, mientras que la interfaz web se desarrolló con React. El proyecto se organizó en cinco sprints, utilizando Scrum como marco de trabajo. Entre sus funcionalidades se encuentra la consulta de partidos, la selección de asientos mediante mapas gráficos de escenarios, la reserva de hasta seis entradas por usuario, la generación de entradas con códigos QR únicos, su validación en el ingreso y la generación de reportes para clientes y administradores. La evaluación incluyó pruebas automatizadas de base de datos, pruebas funcionales de las API, pruebas de concurrencia y una evaluación de accesibilidad. Se ejecutaron 120 pruebas de base de datos sin fallos. En la prueba de concurrencia, 50 usuarios virtuales intentaron reservar el mismo asiento y solo una solicitud consiguió hacerlo. Además, Lighthouse otorgó una puntuación de 100 en accesibilidad. El flujo de extremo a extremo de la API se ejecutó en Apidog con todos sus casos aprobados. Se concluyó que las restricciones de unicidad y los bloqueos pesimistas permiten controlar las reservas simultáneas, mientras que la arquitectura de microservicios facilita la separación de responsabilidades, aunque requiere mecanismos adicionales para integrar y validar información entre servicios.

Palabras clave: microservicios, gestión de entradas, concurrencia, eventos deportivos

Introducción

Los eventos deportivos son una de las actividades de mayor convocatoria en el Ecuador. El fútbol es el caso más representativo: la Liga Profesional de Fútbol del Ecuador (LigaPro) organiza los torneos de la Serie A y la Serie B y publica en su plataforma digital la información de los equipos, los encuentros y los resultados [1]. A los escenarios de fútbol se suman los coliseos, donde se disputan deportes como el baloncesto o el voleibol. En ambos tipos de escenario, los encuentros de mayor interés concentran la demanda de entradas en periodos muy cortos.

La incorporación de plataformas digitales en el entorno del fútbol ecuatoriano ha sido objeto de investigación académica. Altamirano Benítez et al. [2] analizaron las estrategias de comunicación digital que utilizan los clubes de la Serie A. En el ámbito de la venta de entradas, Cayetano Quinde y Flores Villamarín [3] propusieron el diseño de un sistema web y móvil para adquirir entradas a los escenarios de fútbol del país, en el que el usuario consulta los partidos, elige una localidad y obtiene un código único por compra. Estos antecedentes muestran que el acceso digital a las entradas es una necesidad reconocida; sin embargo, el diseño de estas soluciones debe garantizar, además, que un mismo asiento no se venda dos veces cuando muchos aficionados compran al mismo tiempo, independientemente del deporte o del tipo de escenario.

Planteamiento del problema

La venta de entradas para un partido de alta demanda presenta dificultades que afectan tanto al aficionado como al organizador. Cuando varios usuarios intentan comprar el mismo asiento en el mismo instante, un sistema sin control de concurrencia puede venderlo dos veces; si la reserva no tiene un tiempo límite, los asientos quedan bloqueados por compras que nunca se completan; y sin un registro por localidad, el organizador no puede controlar el aforo ni conocer

la ocupación real del escenario. Por ello, resulta pertinente desarrollar un sistema que permita consultar los partidos programados, elegir asientos sobre el plano del escenario y obtener una entrada verificable, con reglas de negocio que impidan la doble venta y la sobreventa.

Justificación

Para cumplir con la escalabilidad, mantenibilidad y desacoplamiento que exige el caso de estudio [4], se eligió trabajar con microservicios. Esto permite que cada propósito del flujo del proyecto se cumpla en un microservicio diferente, creando una dependencia mínima y explícita entre ellos, y esta se maneja mediante contratos REST.

Se trabajó con tres microservicios, auth-service donde se trabaja todo lo referente a autenticación y usuarios; match-service para trabajar escenarios y partidos y reservation-service para reservas y entradas.

En cuanto a la escalabilidad, el microservicio de reservation-service es el que recibe más demanda cuando es un partido de alta convocatoria, por lo que se puede escalar horizontalmente solo ese servicio, sin aumentar recursos a los demás. Aun teniendo varias instancias, si dos usuarios intentan comprar el mismo asiento al mismo tiempo, solo uno lo obtiene, porque la garantía está en una restricción de unicidad de la base(el UNIQUE(match_id, seat_id)), y no la memoria del servicio.

En cuanto a la mantenibilidad, el microservicio más propenso a tener cambios, por el contexto que abarca es el match-service, ya sea por cambio de tipos de escenarios, o incluso tipos de eventos, por ejemplo, agregar un tipo de escenario requiere un valor nuevo en el catálogo de tipos y su dibujo en el plano; el resto del modelo y los demás microservicios no se modifican.

En cuanto al desacoplamiento cada microservicio maneja su propia base de datos y ninguno accede a la base de otro; cuando necesitan comunicarse, lo hacen mediante contratos REST.

Alcance

Este proyecto cuenta con tres perfiles: visitante, cliente y administrador, cada uno tiene sus pantallas y sus permisos controlados.

Para el perfil de visitante, solamente tiene acceso a ver los eventos deportivos y la disponibilidad de asientos de estos.

En cuanto al perfil de cliente, el visitante se registra y se convierte en un cliente, y luego puede:

- Iniciar sesión
- Elegir asientos en el plano y reservar los asientos, contar con 10 minutos (parametrizable) para confirmar
- Confirmar o cancelar la reserva
- Descargar sus entradas en PDF, con un código QR único por asiento
- Revisar sus reservas y cancelar las reservas después de ser confirmadas hasta 24 horas antes del partido

En el proceso de registro, actualmente el correo que llega con la contraseña temporal lo hace mediante Mailpit, esto porque en el entorno de desarrollo se usa este, ya que es un servidor de correo de pruebas.

En cuanto al perfil de administrador, gestiona los escenarios (escenarios y coliseos) con sus secciones y su plano; gestiona los partidos y sus estados; puede activar, desactivar y desbloquear usuarios, y activar y desactivar roles; ver reportes de ocupación y reservas por

evento; además puede cancelar reservas realizadas por parte del cliente, sin límite de 24 horas. También, puede configurar los parámetros del negocio como: el tiempo para confirmar una reserva, el máximo de entradas por cliente, cierre de la venta y el plazo de cancelación.

El administrador no puede realizar reservas, esto con la finalidad de cumplir con la segregación de funciones.

El sistema cuenta con la automatización de inicio y fin de un evento deportivo, esto con la finalidad de gestionar los partidos, el tiempo que por el momento se toma para finalizar un evento deportivo es 120 minutos y este valor se puede ajustar en la configuración del servicio de partidos. Además, el sistema vence automáticamente las reservas pendientes que no se confirman a tiempo y se libera sus asientos, cierra la venta online diez minutos después del inicio del partido y, si un partido se cancela, cancela sus reservas y anula sus entradas.

En cuanto al control de acceso, valida el código QR de las entradas en el ingreso al escenario; cada entrada se usa una sola vez.

Cada partido se ingresa manualmente, actualmente el sistema no consume servicios externos que permita la creación automática de eventos deportivos. Además, los eventos deportivos presentados en el aplicativo solamente son eventos realizados en Ecuador, esto se da porque actualmente el sistema maneja todas las fechas y horas en hora Ecuador (America/Guayaquil).

Actualmente, el sistema no cuenta con: pasarela de pagos, es decir, una vez que se confirma la reserva se da por hecho que el pago fue realizado; con aplicativo móvil, solo es aplicativo Web; solo cuenta con escenarios de coliseos y escenarios, no está contemplado para escenarios al aire libre o piscinas.

Objetivos

Objetivo general

El caso de estudio plantea como objetivo general “Diseñar y desarrollar un Sistema de Gestión de Entradas para eventos deportivos que automatice de forma integral el proceso de reserva de entradas de una organización deportiva. Para ello, se implementará una arquitectura basada en microservicios, aplicando patrones de usabilidad y principios de experiencia de usuario (UX/UI)” [4].

Objetivos específicos

Diseñar la capa de datos del sistema a partir de un modelo entidad-relación (ER) de las entidades del negocio, con scripts SQL de creación, datos semilla y mapeo objeto-relacional (ORM) que garanticen la integridad referencial y la consistencia de la información.

Implementar una arquitectura de microservicios con una base de datos independiente por servicio, comunicación mediante API REST y autenticación basada en JWT, para garantizar la escalabilidad, la mantenibilidad y el desacoplamiento del sistema.

Desarrollar los módulos de administración de usuarios, escenarios deportivos con su distribución de secciones y asientos, y partidos, aplicando un control de acceso basado en los roles Administrador y Cliente.

Implementar el proceso de reserva de asientos para partidos mediante un mapa gráfico con actualización periódica de la disponibilidad, que aplique las reglas de negocio de límite de entradas, tiempo de confirmación y cierre de venta, y emita tickets con un código QR único para el control de acceso.

Generar reportes individuales para el cliente y reportes generales para el administrador, con filtros por partido, fecha y estado, e indicadores de ocupación y disponibilidad.

Validar cumplimiento de los requisitos del sistema mediante pruebas funcionales de API, pruebas de concurrencia, pruebas de base de datos y evaluación de accesibilidad de la interfaz de usuario.

Teoría

Arquitectura de microservicios

Gómez Suárez et al. [5] definen a la arquitectura de microservicios, como un conjunto de servicios pequeños y autónomos que trabajan juntos, es decir, cada servicio cubre una funcionalidad de negocio, esto permite que puedan ser desplegados, evolucionados y escalados de manera independiente. Utilizando este tipo de arquitectura trae como beneficios la escalabilidad, flexibilidad y desacoplamiento.

Barrios Contreras [6] menciona que la diferencia que existe entre tener arquitectura monolítica y arquitectura microservicios es que una arquitectura monolítica es difícil la mantenibilidad de la aplicación, por mas modular que sea la aplicación hacer una modificación por cambios o modificaciones en el modelo de negocio, implica que el código se mas grande y que algunos cambios sean significativos y con el tiempo esto se vuelve mas complejo de mantener. Además, con algunos de estos cambios podría implicar que es necesario incrementar la capacidad de la aplicación, es decir, que el incremento se deberá hacer a toda la aplicación, más no de la funcionalidad que lo requiere. Con la creación e implementación de microservicios, estos puntos antes mencionados se pueden tratar por separado, tanto la modificación y la escalabilidad.

Lewis y Fowler [7] definen a la arquitectura de microservicios como un enfoque en el que una aplicación se desarrolla como un conjunto de servicios pequeños, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros como HTTP. Además, cada microservicio tiene su propia base de datos.

Richardson [8], [9] menciona que cada microservicio es dueño de su propia base de datos, es decir, que ningún otro servicio puede leer o escribir en la base de datos de otros

servicios; para poder comunicarse u obtener información de otro servicio, se lo hace a través de su API. Esta independencia de base de datos se da para que no queden acoplados los servicios y que se pueda desplegar y escalar de manera independiente.

Microsoft [10] define que la comunicación síncrona (HTTP), es cuando el cliente envía la petición y no puede continuar hasta obtener una respuesta, en cambio, la comunicación asíncrona, es cuando el emisor no requiere o no espera una respuesta y cada servicio queda autónomo y disponible, aunque otros estén caídos. También recomienda que la comunicación entre servicios internos debería ser de manera asíncrona y que un servicio no dependa de llamadas síncronas a otro para responder.

Seguridad: autenticación y autorización

Autenticación y autorización

Según OWASP Foundation [11], la autenticación responde a la pregunta "¿Quién eres?", esta se responde realizando el login con usuario y contraseña, en cambio, la autorización responde a la pregunta "¿qué puedes hacer?", en este caso es necesario leer el rol del usuario para entender los permisos asignados a este usuario.

JSON Web Token (JWT)

Jones et al. [12] mencionan que el JWT es un formato compacto para transmitir declaraciones (claims) entre dos partes, estas están representadas como un objeto JSON y protegidas mediante una firma digital. El token es un texto que está compuesto por tres partes el encabezado, el contenido con las declaraciones y la firma. Los datos no están cifrados, solo codificados en Base64. Cualquiera puede leerlos en jwt.io. Lo que no se puede hacer es modificarlo, porque la firma dejaría de coincidir.

Este mecanismo permite que la autenticación sea sin estado (stateless), es decir, el servidor no necesita almacenar sesiones, porque el token contiene la información necesaria y su firma confirma el origen, hablando desde la arquitectura de microservicios este mecanismo es favorable porque cada servicio puede validar el token de forma independiente sin necesidad de consultar al servicio de autenticación en cada petición.

Para la firma se empleó el algoritmo HS256, que corresponde a HMAC con la función hash SHA256 [13]. Esta firma se trata de un algoritmo simétrico, es decir, que la misma clave secreta se utiliza para firmar y para verificar el token, por lo que esta calve debe compartirse entre todos los servicios que validan este token

Almacenamiento de contraseñas

Las contraseñas no deberían cifrarse, sino realizar un proceso de hash. El cifrado es una operación reversible, es decir, que permite recuperar el valor original utilizando la clave correspondiente. Utilizando hash, es una función de una sola vía [14]. Con esto se obtiene que la verificación del inicio de sesión, el sistema calcule el hash de la contraseña ingresada y se compare con la contraseña almacenada, sin necesidad de conocer la contraseña original.

Las funciones hash de propósito general, como lo es SHA-256, resultan inadecuadas para este fin porque su velocidad facilita los ataque de fuerza bruta, por esta razón, BCrypt fue diseñado especialmente para contraseñas. Incorpora una sal aleatoria por cada contraseña, lo que impide el uso de tablas precalculadas. También incorpora un factor de costo ajustable, que permite aumentar el tiempo de cálculo a medida que el hardware se vuelve más potente [15]. OWASP recomienda configurarlo con un factor de trabajo mínimo de 10 [14].

Transacciones y concurrencia

Las transacciones son un conjunto de operaciones que se ejecutan como una unidad lógica de trabajo sobre una base de datos. La finalidad de las transacciones es preservar la integridad y consistencia de la información ante errores o solicitudes simultaneas.

Las propiedades ACID describen cuatro características fundamentales de las transacciones. La atomicidad que garantiza que las operaciones de una transacción se completen en conjunto o se reviertan, la consistencia asegura que se respeten las restricciones de integridad; el aislamiento controla los efectos de las transacciones concurrentes; y la durabilidad garantiza la persistencia de los cambios [16].

Cuando el resultado de una operación depende del orden o del momento en que se ejecutan varias operaciones concurrentes, esto se conoce como una condición de carrera (race condition), es decir, que cuando dos usuarios intentan reservar simultáneamente la misma localidad para un evento deportivo, ambos podrían recibir inicialmente una respuesta de disponibilidad. Sin mecanismos adecuados de control, el sistema podría aceptar operaciones incompatibles.

Para controlar estos escenarios, se puede utilizar dos estrategias, el bloqueo pesimista y optimista. Si hablamos del bloqueo pesimista, decimos que impide temporalmente que otras transacciones modifiquen un recurso mientras se ejecuta una operación critica. El bloqueo optimista, en cambio permite que las operaciones avancen y comprueba, antes de confirmar los cambios, si el cambio fue modificado por otra transacción.

Otra de las defensas que se puede aplicar es poner restricciones de integridad en la base de datos. Una restricción de tipo UNIQUE impide duplicar valores de una columna o combinaciones de columnas protegidas.

Diseño de APIs REST

REST (*Representational State Transfer*) es un estilo arquitectónico para diseñar sistemas distribuidos mediante recursos identificados por URI y una interfaz uniforme de comunicación. Entre sus restricciones se encuentran la separación cliente-servidor, la comunicación sin estado (*stateless*), la capacidad de almacenamiento en caché cuando corresponda, la interfaz uniforme y la arquitectura por capas [17].

En una aplicación de eventos deportivos, una API REST puede ofrecer recursos para consultar eventos, disciplinas deportivas, recintos y opciones de entradas. Los métodos HTTP permiten expresar las operaciones realizadas sobre estos recursos: GET consulta información; POST crea recursos o ejecuta operaciones definidas por el servidor; PUT reemplaza un recurso; PATCH modifica parcialmente un recurso; y DELETE solicita su eliminación.

Los códigos de estado HTTP permiten comunicar el resultado de cada solicitud. El código 201 Created indica que se creó un recurso; 400 Bad Request representa una solicitud incorrecta; 401 Unauthorized indica que las credenciales no son válidas o no se han proporcionado; 403 Forbidden señala que la operación no está autorizada; 404 Not Found indica que el recurso solicitado no se encuentra; 409 Conflict representa un conflicto con el estado actual del recurso; 422 Unprocessable Content identifica contenido que no puede procesarse por razones semánticas; y 503 Service Unavailable señala que el servicio no puede atender temporalmente la solicitud.

El uso de estos códigos debe ser coherente con el contrato de la API. Por ejemplo, 409 Conflict podría utilizarse cuando una localidad ya está reservada, siempre que el sistema gestione directamente las reservas.

OpenAPI 3 [18] es una especificación que permite describir formalmente una API HTTP, incluyendo sus rutas, parámetros, esquemas de datos, respuestas y mecanismos de autenticación. Este contrato facilita la documentación, la generación de clientes y la elaboración de pruebas automatizadas.

Por su parte, RFC 9457 [19] define un formato estandarizado para representar errores HTTP mediante documentos estructurados, generalmente con el tipo de contenido application/problem+json. Sus campos pueden incluir type, title, status, detail e instance, con el propósito de comunicar el tipo de error y proporcionar información útil para el cliente.

Aplicaciones web de una sola página

Una aplicación web de una sola página (*Single Page Application*, SPA) es una aplicación que carga un documento inicial y actualiza dinámicamente su contenido durante la navegación, sin recargar el documento HTML completo en cada interacción. React es una biblioteca de JavaScript que permite construir interfaces mediante componentes reutilizables y gestionar la representación de los datos en la interfaz [20].

En una aplicación de eventos deportivos, una SPA puede presentar vistas para explorar eventos, consultar sus detalles y acceder a las opciones de entradas. El enrutamiento del cliente permite asociar diferentes direcciones URL con los componentes correspondientes, por ejemplo, una vista de eventos, el detalle de un evento o una sección de entradas. Una biblioteca como React Router puede gestionar estas rutas.

Durante el desarrollo, el frontend puede ejecutarse en un origen distinto al backend. El origen está determinado por el protocolo, el dominio y el puerto. Por ello, una aplicación en http://localhost:5173 y una API en http://localhost:8080 tienen orígenes diferentes, aunque ambas utilicen localhost.

CORS (*Cross-Origin Resource Sharing*) es un mecanismo de seguridad implementado por los navegadores que controla el acceso a recursos de otros orígenes [21]. Si el servidor no permite el origen del frontend mediante las cabeceras correspondientes, el navegador puede impedir que JavaScript acceda a la respuesta.

Vite [22] permite configurar un proxy de desarrollo para reenviar las solicitudes del frontend al backend. De esta manera, el navegador realiza las peticiones a través del origen del servidor de desarrollo, evitando el problema de comunicación entre orígenes durante ese entorno. El proxy no elimina CORS ni sustituye la configuración de seguridad necesaria en producción.

Calidad y pruebas de software

La calidad del software comprende las características que permiten determinar si un producto cumple sus requisitos y satisface las necesidades para las cuales fue desarrollado. La ingeniería de software incorpora actividades sistemáticas de análisis, diseño, construcción, verificación, validación y mantenimiento para gestionar la calidad durante el ciclo de vida del producto [23].

Las pruebas unitarias examinan componentes individuales, como funciones de validación o componentes de interfaz. Las pruebas de integración comprueban las interacciones entre componentes, como la comunicación entre el frontend y una API. Las pruebas de sistema evalúan el sistema integrado frente a sus requisitos, mientras que las pruebas de extremo a extremo (*End-to-End*, E2E) recorren flujos completos desde la perspectiva del usuario.

La pirámide de pruebas representa una estrategia que suele priorizar una base amplia de pruebas unitarias, un nivel intermedio de pruebas de integración y un conjunto menor de pruebas E2E. Esta distribución busca obtener retroalimentación rápida y reducir los costes de ejecución y mantenimiento, aunque debe adaptarse a la arquitectura y los riesgos de cada proyecto.

En una aplicación de eventos deportivos, las pruebas de API permiten comprobar la estructura de las respuestas, los códigos HTTP, las validaciones y la conformidad con los contratos documentados. Las pruebas de carga permiten analizar el rendimiento bajo diferentes volúmenes de solicitudes, mientras que las pruebas de concurrencia examinan el comportamiento de operaciones simultáneas y detectan posibles conflictos de estado. Estas últimas son especialmente relevantes cuando el sistema administra reservas o compras; si la aplicación solo redirige a proveedores externos, su alcance de pruebas debe reflejar esa limitación.

El análisis estático de código complementa las pruebas dinámicas al examinar el código sin ejecutarlo. Puede identificar problemas potenciales, complejidad excesiva, duplicación y otros defectos de mantenibilidad. Herramientas como SonarQube permiten incorporar estos análisis a los procesos de integración continua.

Finalmente, es importante distinguir entre verificación y validación. La verificación determina si el producto cumple las especificaciones establecidas, mientras que la validación evalúa si satisface las necesidades y el uso previsto. En este proyecto, comprobar que una API devuelve la estructura definida en OpenAPI es un ejemplo de verificación; evaluar si los usuarios pueden encontrar un evento y acceder fácilmente a sus opciones de entradas es un ejemplo de validación.

Marco de trabajo Scrum

Scrum es un marco de trabajo ágil que permite desarrollar productos complejos mediante ciclos iterativos e incrementales. Se fundamenta en el empirismo y el pensamiento Lean, y utiliza la transparencia, la inspección y la adaptación para gestionar el trabajo y mejorar continuamente el producto [24].

Scrum define tres responsabilidades dentro del Scrum Team: el Product Owner, responsable de maximizar el valor del producto y gestionar eficazmente el Product Backlog; el Scrum Master, responsable de establecer Scrum y mejorar la efectividad del equipo; y los Developers, responsables de crear el incremento del producto y realizar el trabajo necesario para mantener su calidad.

Los eventos de Scrum comprenden el Sprint, Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective. El Sprint constituye el periodo de duración fija durante el cual se desarrolla un incremento. La planificación establece el objetivo y el trabajo previsto; la reunión diaria permite inspeccionar el progreso y adaptar el plan; la revisión permite examinar el resultado con los interesados; y la retrospectiva identifica oportunidades de mejora en el proceso de trabajo.

Scrum utiliza tres artefactos: el Product Backlog, que contiene el trabajo ordenado para mejorar el producto; el Sprint Backlog, que reúne el objetivo del Sprint, los elementos seleccionados y el plan de trabajo; y el Increment, que representa el resultado integrado que cumple la Definition of Done. Cada artefacto tiene un compromiso asociado: Product Goal, Sprint Goal y Definition of Done, respectivamente.

Una historia de usuario expresa una necesidad desde la perspectiva del usuario y suele formularse mediante la estructura: «Como [tipo de usuario], quiero [funcionalidad], para [beneficio]». Por ejemplo: «Como persona interesada en eventos deportivos, quiero consultar eventos de distintas disciplinas en Ecuador, para identificar las actividades a las que me gustaría asistir».

Los puntos de historia son una técnica de estimación relativa que algunos equipos utilizan para representar el esfuerzo, la complejidad y la incertidumbre de los elementos del Product Backlog. No equivalen directamente a horas y no son obligatorios en Scrum.

La Definition of Done establece las condiciones de calidad que debe cumplir un incremento para considerarse terminado. En esta aplicación, el equipo podría incluir criterios como la ejecución satisfactoria de las pruebas requeridas, la integración del código, el cumplimiento de los criterios de aceptación y la actualización de la documentación pertinente.

Metodología experimental

El presente trabajo corresponde a un desarrollo tecnológico aplicado, en el que se construyó y validó un sistema de software a partir de los requerimientos del caso de estudio [4]. Para su desarrollo se adaptó el marco de trabajo Scrum [24] a un equipo de una persona y se implementó una arquitectura de tres microservicios, cada uno con su propia base de datos. La calidad del sistema se verificó mediante pruebas de base de datos, pruebas funcionales de extremo a extremo de la API con Apidog, pruebas de concurrencia con k6 y una evaluación de accesibilidad de la interfaz con Lighthouse. En esta sección se describen la metodología de desarrollo, la arquitectura de la solución, el modelo de datos, las reglas de negocio implementadas, las herramientas utilizadas, la estrategia de pruebas y el uso de herramientas de inteligencia artificial.

Durante el desarrollo se utilizó un asistente de inteligencia artificial como apoyo en la implementación y la documentación, cuyo alcance se detalla al final de esta sección.

Metodología de desarrollo

Al tratarse de un equipo de una sola persona, los tres roles definidos por Scrum [24] (Product Owner, Scrum Master y Developer) fueron asumidos por la autora. Como Product Owner, se priorizó el backlog a partir de los requisitos del caso de estudio [4]; como Developer, se implementó y probó cada incremento. La planificación y el seguimiento se registraron en el plan de sprints (ver Anexo A).

El desarrollo se organizó en cinco sprints, como se muestra en la Tabla 1. El Sprint 0 estableció la base del proyecto: el entorno de desarrollo y la base de datos. En el Sprint 1 se implementaron los microservicios de partidos y de autenticación; en el Sprint 2, el microservicio

de reservas; en el Sprint 3, el frontend; y el Sprint 4 se destinó al cierre, con las pruebas de extremo a extremo, la documentación y la entrega.

Tabla 1

Planificación de los sprints del proyecto

Sprint	Fechas	Objetivo (Sprint Goal)	HU	Puntos
0	17 oct.	Base de datos y entorno	4	13
1	18–20 oct.	Microservicios de partidos y de autenticación	10	31
2	20–22 oct.	Microservicio de reservas	10	43
3	22–24 oct.	Frontend	10	37
4	25–26 oct.	Cierre: pruebas E2E, documentación y entrega	13	42
Total	**17–26 oct.**		**47**	**166**

Nota. Elaboración propia a partir del plan de sprints (Anexo A). HU: historias de usuario; Puntos: puntos de historia planificados.

Las historias de usuario se definieron a partir de los requisitos del caso de estudio [4], con el formato “Como [rol], quiero [funcionalidad], para [beneficio]”; por ejemplo, la HU-16: “Como cliente, quiero reservar hasta 6 asientos por partido para asistir con acompañantes”. Cada historia incluyó criterios de aceptación verificables y una estimación en puntos de historia, y se clasificó como funcional o técnica. En total se definieron 47 historias, con 166 puntos.

Durante el desarrollo surgieron ocho historias nuevas (HU-40 a HU-47), identificadas al probar el sistema. Algunas respondían a reglas de negocio implícitas que el caso de estudio no detallaba, como el cierre automático de la venta, el cambio de estado de los partidos según su horario, el plazo de cancelación y la anulación de entradas de un partido cancelado. Otras correspondían a requisitos del caso de estudio que no se habían cubierto inicialmente, como el registro de coliseos. Además, se incorporó una pantalla para que el administrador configure los

parámetros del negocio sin modificar el código. La incorporación de estas historias aplicó en la práctica el principio de inspección y adaptación del empirismo de Scrum [24].

El sistema se construyó en el siguiente orden. Primero se creó la base de datos y se cargaron los datos semilla, para garantizar la integridad de los datos iniciales y el cumplimiento de las restricciones definidas. Después se desarrolló el backend, de modo que las reglas de negocio se validaran mediante pruebas de API antes de construir la interfaz. Finalmente, se desarrolló el frontend, que presenta al usuario la información gestionada por los servicios.

Para el control de versiones se utilizó Git y GitHub, con una rama por historia de usuario (feature/HU-xx-…) y un pull request para integrar cada cambio en la rama principal. A la fecha, el repositorio cuenta con 141 commits y 30 pull requests integrados.

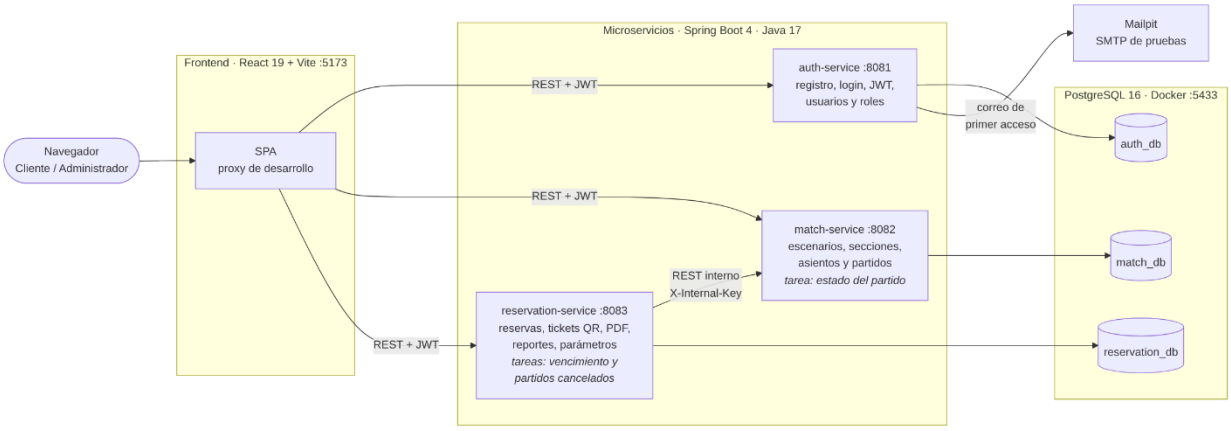
Una historia de usuario se consideró terminada cuando cumplía sus criterios de aceptación, sus pruebas eran satisfactorias, el contrato OpenAPI estaba actualizado (cuando aplicaba) y el cambio se había integrado en la rama principal mediante un pull request.

Arquitectura de la solución

La arquitectura del sistema está compuesta por tres microservicios, separados por capacidad de negocio, cada uno con su propia base de datos, y un frontend de página única (SPA) desarrollado en React, como se observa en la Figura 1.

Figura 1

Arquitectura de componentes del sistema



Nota. Elaboración propia

El microservicio auth-service (puerto 8081) se encarga del registro de usuarios, el inicio de sesión, la emisión del JWT y la gestión de usuarios y roles; sus datos se almacenan en auth_db. El microservicio match-service (puerto 8082) gestiona los escenarios, las secciones, los asientos y los partidos, incluido el cambio automático del estado de los partidos; sus datos se almacenan en match_db. Finalmente, el microservicio reservation-service (puerto 8083) gestiona las reservas, los tickets con código QR, los PDF, los reportes, los parámetros del negocio y el control de acceso, además del vencimiento de reservas y la anulación de entradas de partidos cancelados; sus datos se almacenan en reservation_db.

La comunicación entre el frontend y los microservicios se realiza mediante API REST, con el JWT en el encabezado Authorization. Cada servicio valida el token de forma independiente, sin consultar al servicio de autenticación. Además, existe una comunicación interna síncrona entre reservation-service y match-service, protegida con la clave X-Internal-Key, que permite validar que los asientos existan y pertenezcan al escenario del partido, consultar los datos del partido y obtener los nombres de los asientos para el PDF.

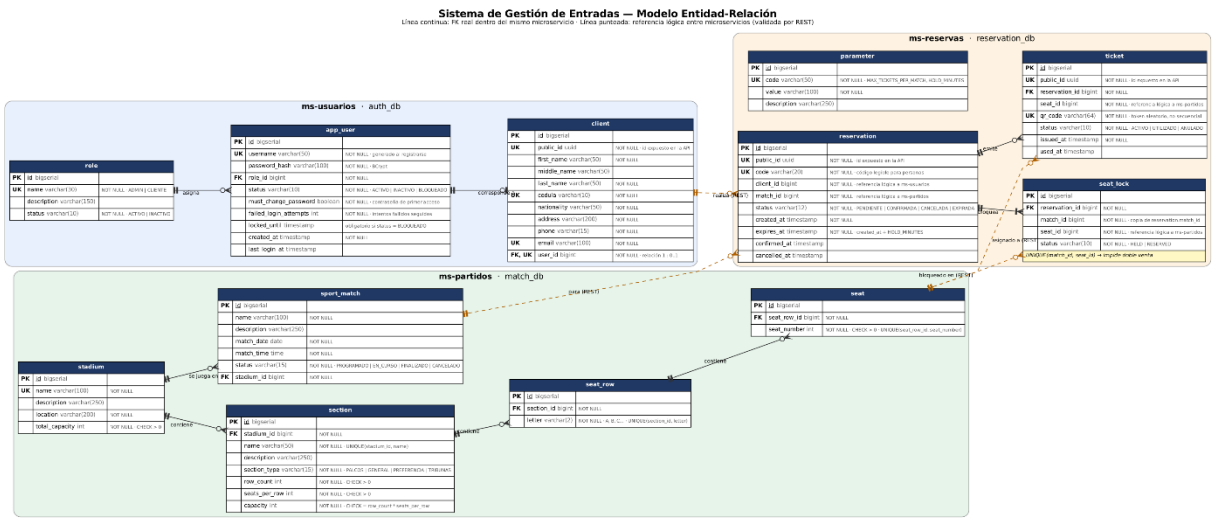
Como cada microservicio tiene su propia base de datos, no existen llaves foráneas entre ellas; los identificadores que pertenecen a otro servicio se validan mediante consultas REST, de acuerdo con el patrón de una base de datos por servicio [8]. Cuando una pantalla necesita datos de varios servicios, el frontend los combina; por ejemplo, el reporte del administrador une las reservas de reservation-service con los nombres de los clientes de auth-service. Si alguno de los servicios consultados no responde, el sistema degrada su funcionamiento en lugar de fallar: el historial de reservas y las entradas en PDF se muestran sin los datos del partido, y el reporte muestra el identificador del cliente en lugar de su nombre. La consistencia entre servicios se mantiene, además, mediante tres tareas programadas: el cambio de estado de los partidos, el vencimiento de las reservas no confirmadas y la anulación de las entradas de partidos cancelados. En el entorno de desarrollo, las tres bases de datos se ejecutan en una misma instancia de PostgreSQL 16 en Docker, y el envío de correos se simula con Mailpit, un servidor SMTP de pruebas.

Modelo de datos

Para la construcción de la base de datos se utilizó PostgreSQL 16, en modelo de datos que se diseñó se hizo a partir de un modelo entidad-relación. Se hizo una base de datos por cada microservicio, es decir, se tiene 3 base de datos (auth_db, match_db y reservation_db) (Figura 2) y un total de 12 tablas.

Figura 2

Modelo entidad-relación de las tres bases de datos



Nota. Elaboración propia.

Para organizar la información del sistema, se decidió que cada servicio administre su propia base de datos. Por esta razón, cuando una reserva necesita relacionarse con un cliente, un partido o un asiento, se utilizan identificadores como client_id, match_id y seat_id. Estas referencias se validan mediante consultas REST al servicio responsable de cada entidad, ya que no se utilizan llaves foráneas entre bases de datos diferentes. En cambio, las relaciones que pertenecen a una misma base de datos sí se controlan mediante llaves foráneas. Además, se utiliza ON DELETE RESTRICT en las relaciones donde es necesario conservar información histórica, evitando que se eliminen registros que todavía están relacionados con otros datos, como las entradas vendidas.

En las tablas se utilizan identificadores internos de tipo BIGINT para relacionar los registros. Por otra parte, los clientes, las reservas y las entradas cuentan con identificadores públicos de tipo UUID, que son los que se exponen en las URL de la API. Esta separación

permite mantener identificadores internos sencillos para las relaciones y utilizar identificadores públicos menos predecibles, sin exponer directamente la secuencia de registros de la base de datos.

También fue necesario considerar que un asiento no puede tener un único estado para todos los eventos, porque puede estar ocupado en una competición y disponible en otra. Para manejar esta situación, se creó la tabla seat_lock, que registra la relación entre el asiento y el partido en el que se encuentra reservado. La restricción UNIQUE(match_id, seat_id) evita que se registren dos asignaciones para el mismo asiento y partido. De esta manera, la propia base de datos ayuda a prevenir reservas duplicadas cuando varias solicitudes se procesan al mismo tiempo. Para que el asiento pueda volver a estar disponible después de una cancelación o de que expire una reserva, también deben aplicarse las reglas correspondientes de liberación.

Los parámetros que pueden cambiar según las necesidades del negocio, como el máximo de entradas permitido por partido o el tiempo disponible para confirmar una reserva, se almacenan en la tabla parameter. Así, estos valores pueden modificarse desde la configuración de la base de datos, sin tener que cambiar el código cada vez que se ajusta una regla.

Finalmente, los scripts de creación de la base de datos incluyen restricciones con nombres definidos, índices para facilitar las consultas y una función para normalizar los nombres de escenarios y secciones. Esta función permite detectar nombres que, aunque estén escritos con diferencias de mayúsculas, tildes o espacios, corresponden al mismo nombre normalizado. Para las pruebas se prepararon datos de ejemplo que representan cinco escenarios y un coliseo, distribuidos en 41 secciones y con un total de 44 440 asientos. También se incluyeron usuarios, partidos y reservas en distintos estados, con el propósito de probar diferentes situaciones del sistema.

Reglas de negocio implementadas

La Tabla 2 presenta las principales reglas de negocio del sistema y la forma en que se controla su cumplimiento. Estas reglas se validan en el backend, mientras que el frontend también comprueba algunas de ellas para mostrar advertencias al usuario antes de enviar la solicitud. Sin embargo, la validación del frontend no reemplaza la del backend, ya que las solicitudes deben cumplir las reglas del sistema independientemente de cómo se realicen.

Tabla 2

Reglas de negocio y mecanismo que las garantiza

Regla	Mecanismo	Respuesta
Máximo 6 entradas por cliente y partido	Parámetro MAX_TICKETS_PER_MATCH	422
Un asiento no se vende dos veces	Consulta previa + UNIQUE(match_id, seat_id)	409
Retención de asientos por 10 minutos	HOLD_MINUTES + tarea programada	EXPIRADA
Cierre de la venta online	Inicio + SALES_CLOSE_OFFSET_MINUTES (10)	422
Inicio y fin automático del partido	Tarea programada cada minuto (120 min)	—
Bloqueo por intentos fallidos	5 intentos → BLOQUEADO 15 min	423
Solo se ven reservas propias	Id del cliente desde el token (anti-IDOR)	404
Cancelación del cliente hasta 24h antes del partido	Parámetro CANCEL_DEADLINE_HOURS	422
Partido cancelado	Tarea programada que cancela las reservas y anula las entradas	ANULADA
Cada entrada se usa una sola vez	Validación QR con bloqueo pesimista	YA_UTILIZADA
El Administrador no reserva	Autorización por rol	403

Una de las reglas más importantes es evitar que un mismo asiento se venda dos veces para un mismo evento. Para ello, se implementaron dos controles. Primero, el servicio consulta si el asiento está disponible antes de intentar reservarlo, lo que permite informar al usuario si ya fue ocupado. Además, la base de datos cuenta con una restricción UNIQUE(match_id, seat_id), que impide registrar dos veces la misma combinación de partido y asiento. Este segundo control es necesario porque dos solicitudes pueden consultar la disponibilidad al mismo tiempo y recibir una respuesta positiva antes de que alguna registre la reserva. Esta situación se conoce como condición de carrera, y demuestra por qué la consulta inicial no es suficiente para evitar reservas duplicadas [16].

También se utiliza bloqueo pesimista en las operaciones que modifican el estado de una reserva o de una entrada, como la confirmación, cancelación, expiración y validación del código QR. Con este mecanismo se evita que dos operaciones concurrentes modifiquen el mismo registro de forma incompatible. Por ejemplo, si un código QR se presenta simultáneamente en dos puntos de acceso, el sistema debe asegurar que la entrada se valide una sola vez.

Las reglas relacionadas con los tiempos se calculan a partir de la fecha y hora programadas para el partido, en lugar de depender únicamente del estado registrado por el administrador. De esta manera, la venta se cierra diez minutos después del inicio del partido, aunque su estado no se haya actualizado manualmente. De igual forma, una reserva pendiente mantiene los asientos retenidos durante un máximo de diez minutos o hasta el cierre de la venta, lo que ocurra primero. Estos tiempos pueden ajustarse mediante parámetros configurables por el administrador, siempre dentro de los rangos permitidos por el sistema.

Herramientas y tecnologías

Tabla 3

Herramientas utilizadas

Herramienta	Versión	Uso en el proyecto
Java / Spring Boot	17 / 4.1.1	Microservicios
PostgreSQL (Docker)	16	Bases de datos
React / Vite8	19 / 8	Interfaz web
Apidog	2.8.51	Diseño y pruebas de API
k6	v1.1.0	Pruebas de carga y concurrencia
DBeaver	26.2.2	Consulta de datos
Git / GitHub	—	Control de versiones y pull requests
Claude (Anthropic)	—	Asistente de IA (ver sección de uso de IA)
Lighthouse | [versión de Chrome]	---	Evaluación de accesibilidad

La Tabla 3 presenta todas las herramientas y tecnologías utilizadas en el desarrollo y la validación del sistema, junto con su versión y función en el proyecto.

Estrategia de pruebas

Se definió una estrategia de pruebas basada en riesgos, dando prioridad a los componentes en los que se concentran los problemas más críticos del sistema: la doble venta de un asiento, la integridad de los datos y la comunicación entre los microservicios. Por esta razón, las pruebas se enfocaron principalmente en la base de datos, las API y el comportamiento del sistema ante solicitudes simultáneas.

Pruebas de base de datos**.** Se implementaron 120 casos automatizados en SQL. De estos, 80 comprueban que las restricciones de integridad rechacen operaciones no permitidas, mientras que los 40 restantes verifican la consistencia de los datos semilla. Además, se

realizó una prueba de mutación manual: se eliminó intencionalmente la restricción encargada de impedir la doble venta de un asiento y se comprobó que la suite detectara el cambio mediante un fallo.

Pruebas funcionales de extremo a extremo de las API**.** Se ejecutaron 56 casos en Apidog, distribuidos en nueve escenarios. Las pruebas comprobaron los códigos HTTP y el contenido de las respuestas en los flujos principales, así como en situaciones negativas y casos relacionados con la seguridad.

Pruebas de concurrencia y carga. Se utilizó k6 para simular 50 usuarios virtuales que intentaron reservar simultáneamente el mismo asiento. También se realizaron consultas de disponibilidad durante 30 segundos, con 20 usuarios virtuales, para observar el comportamiento del sistema ante solicitudes concurrentes.

Evaluación de accesibilidad. Se utilizó Lighthouse para evaluar las pantallas principales de la interfaz y obtener indicadores relacionados con su accesibilidad.

Pruebas exploratorias. Se realizaron recorridos manuales por la interfaz utilizando los dos roles definidos en el sistema. Durante estas pruebas se identificaron los defectos que se presentan en la Tabla 6.

El código también se estructuró para facilitar la incorporación de pruebas unitarias. Para ello, se utilizaron la inyección de dependencias por constructor y la separación de las reglas de negocio en clases pequeñas. Sin embargo, las pruebas unitarias no formaron parte de las prioridades de esta etapa, debido a que, aunque permiten comprobar la lógica de manera aislada, no sustituyen las pruebas que requieren una base de datos real ni aquellas que evalúan la integración y la concurrencia entre componentes. Su incorporación se considera una mejora para futuras iteraciones.

Las pruebas se ejecutaron en el equipo de desarrollo, con las siguientes características: procesador CORE(TM) i7-1188H, 16 GB DDR de memoria RAM, Windows 11 Home, Docker Desktop 4.97.0 y PostgreSQL 16. Los tres microservicios se ejecutaron localmente mediante el entorno de desarrollo. Como datos de prueba se utilizaron los registros semilla, que incluyen clientes con reservas en distintos estados, una cuenta inactiva, una cuenta bloqueada y partidos programados, finalizados y cancelados.

Uso de herramientas de inteligencia artificial

Para el desarrollo del sistema se utilizó Claude, una herramienta de inteligencia artificial desarrollada por Anthropic, como apoyo principal en la generación del código fuente del backend y del frontend, los scripts de base de datos, las pruebas automatizadas y la documentación técnica. También se utilizó para apoyar la redacción de algunas secciones del presente informe.

La participación de la autora se centró en definir los requisitos y las historias de usuario a partir del caso de estudio, orientar el desarrollo de las funcionalidades, revisar el comportamiento del sistema, identificar defectos y necesidades no cubiertas, y decidir qué ajustes debían implementarse. A partir de las pruebas y de la revisión funcional se identificaron problemas que dieron lugar a las historias de usuario HU-40 a HU-47 y a las correcciones documentadas en la Tabla 6. Asimismo, la autora revisó e integró los cambios realizados y gestionó el control de versiones mediante *pull requests*.

El contenido generado con ayuda de la herramienta se revisó durante el proceso de desarrollo y se sometió a las pruebas correspondientes según la estrategia definida para el proyecto. Cuando las propuestas no respondían a los requisitos o al comportamiento esperado, se

realizaron ajustes o se descartaron. La autora asumió la responsabilidad de revisar los resultados, tomar las decisiones de implementación y evaluar el funcionamiento del sistema obtenido.

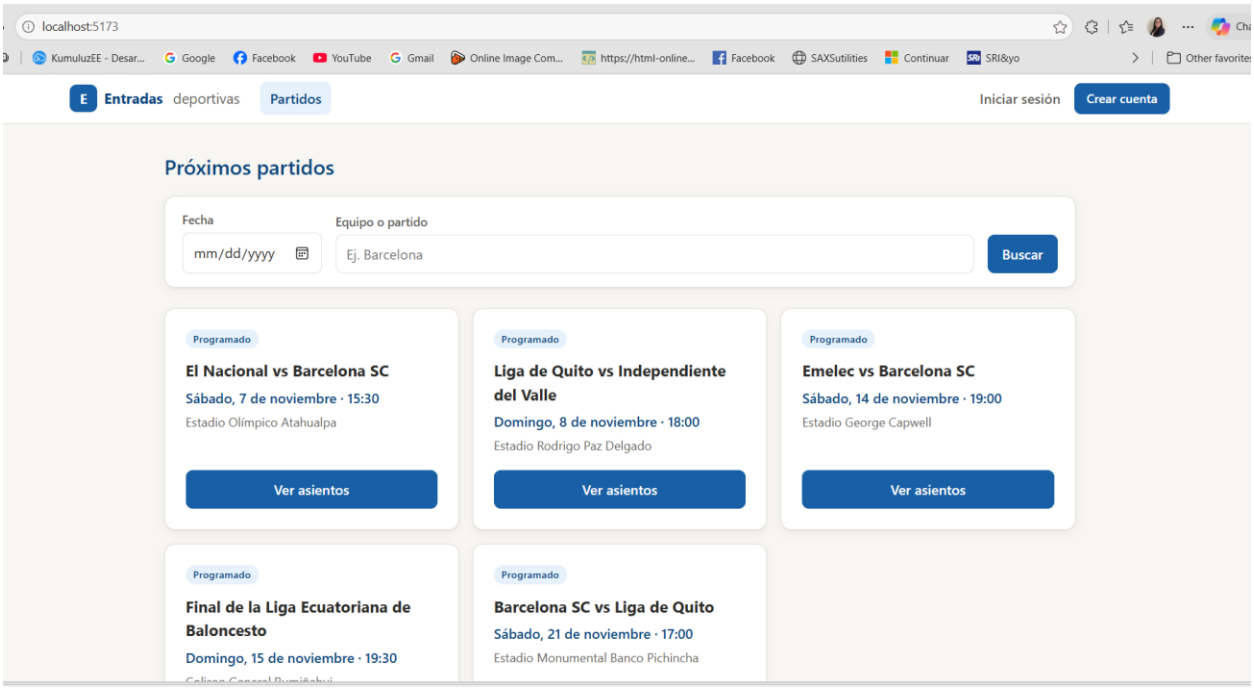
Análisis y resultados

Funcionalidades implementadas

De las 47 historias de usuario planificadas, 41 se completaron, con 149 de 166 puntos de historia (90 %). Las historias pendientes corresponden a la ejecución final de las pruebas, la documentación de entrega y tres mejoras que se trasladaron a trabajo futuro (HU-35, HU-36 y HU-39). Las figuras 3 a 12 muestran las pantallas principales del sistema.

Figura 3

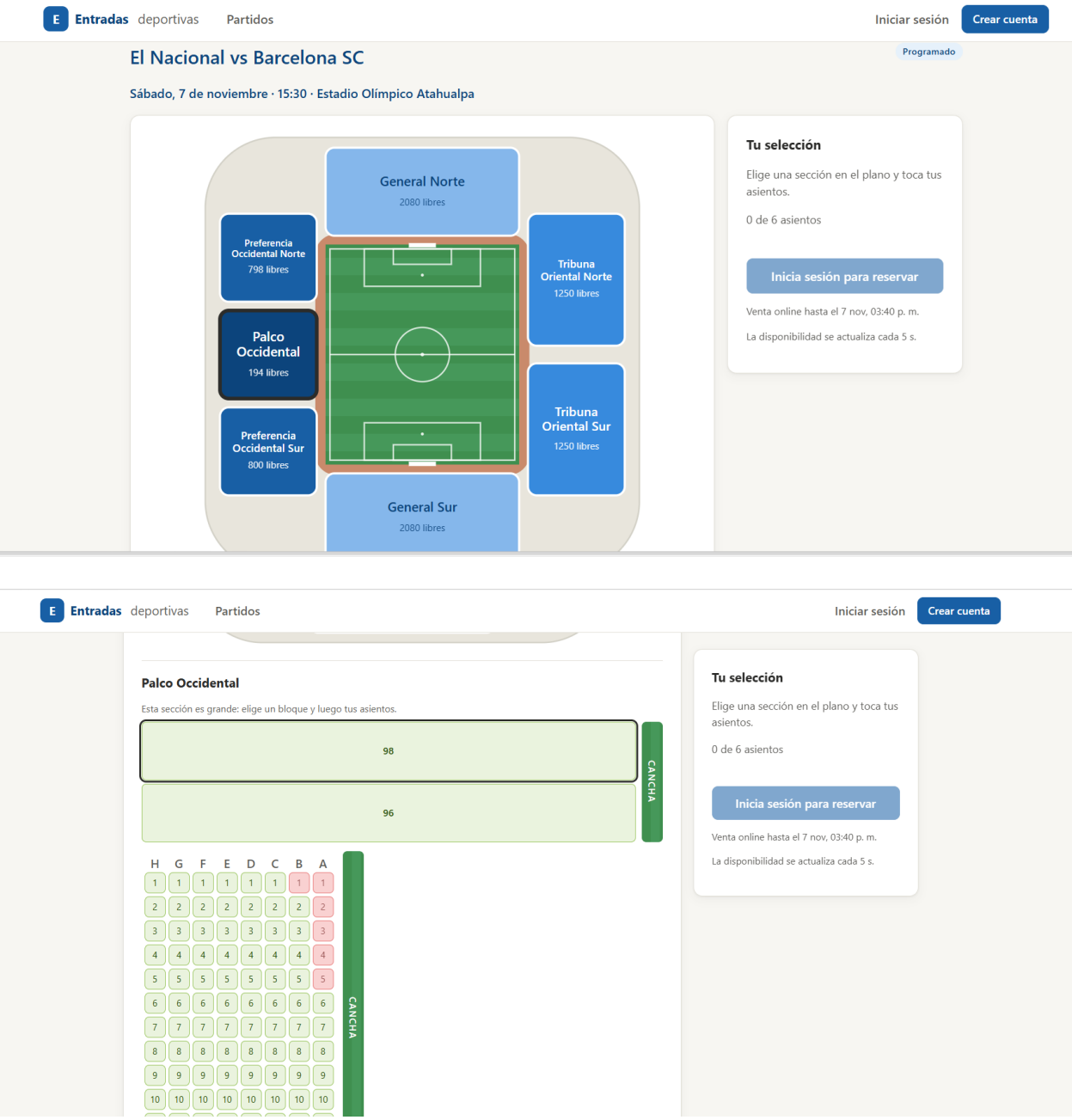
Catálogo de partidos



Nota. Captura de pantalla del sistema.

Figura 4

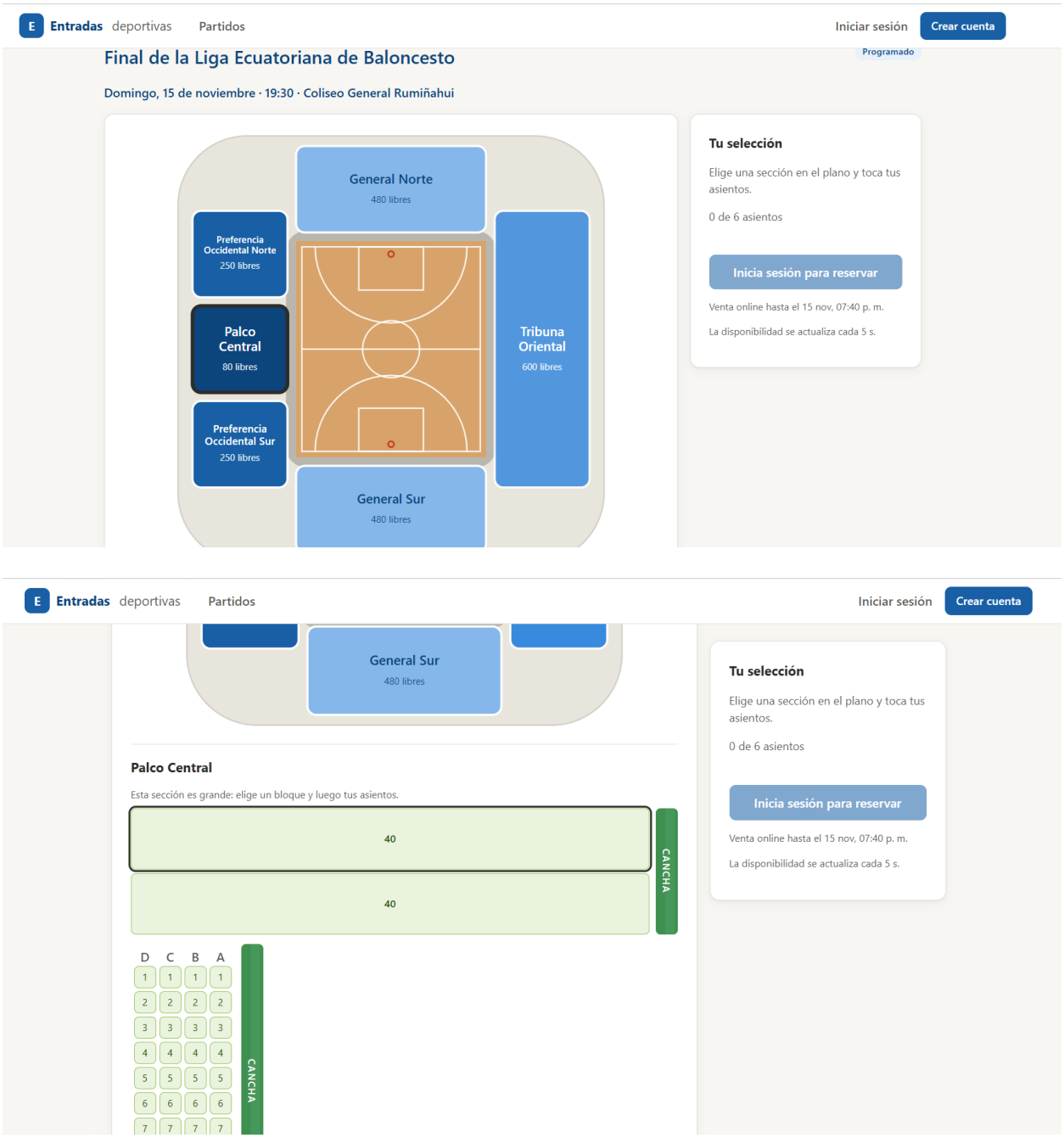
Mapa de asientos de un escenario



Nota. Captura de pantalla del sistema.

Figura 5

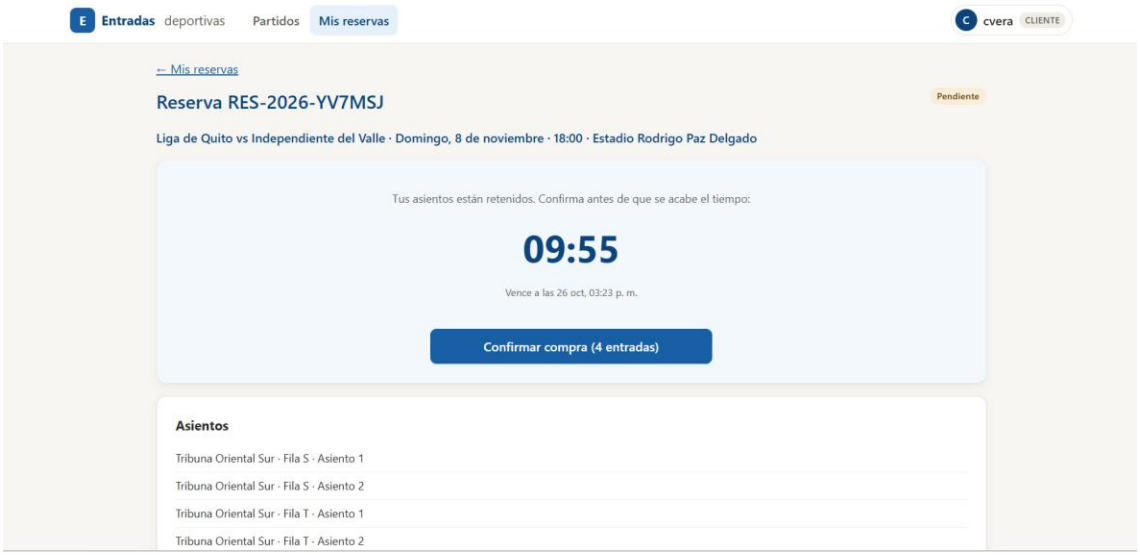
Mapa de asientos de un coliseo



Nota. Captura de pantalla del sistema.

Figura 6

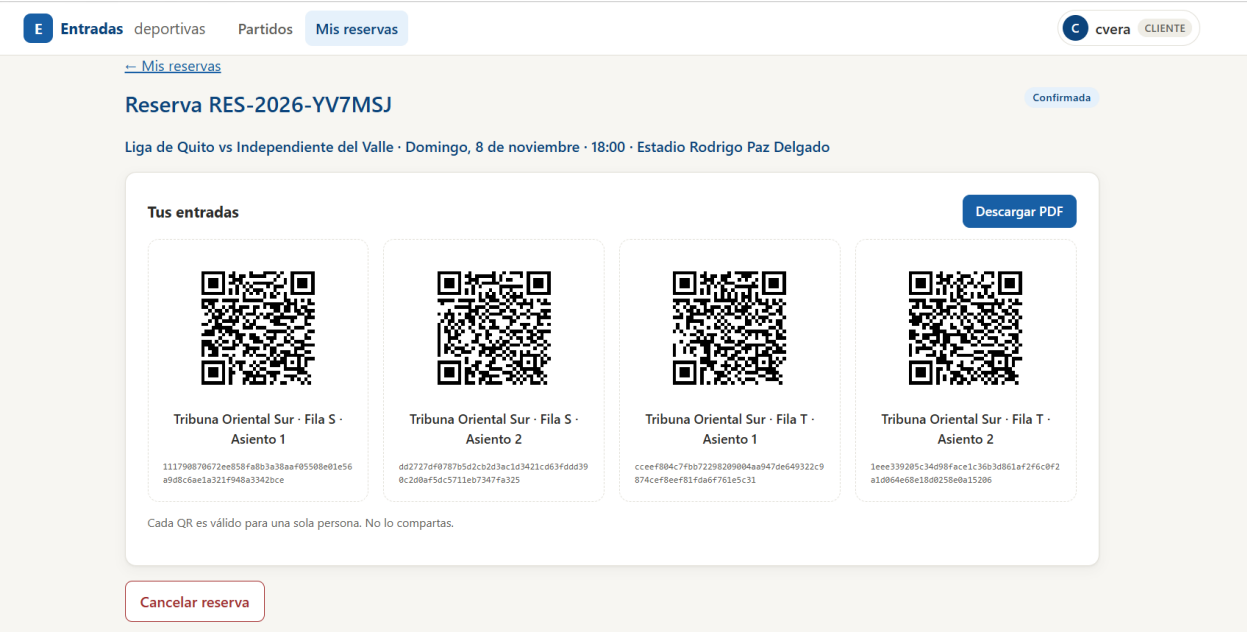
Reserva con cuenta regresiva



Nota. Captura de pantalla del sistema.

Figura 7

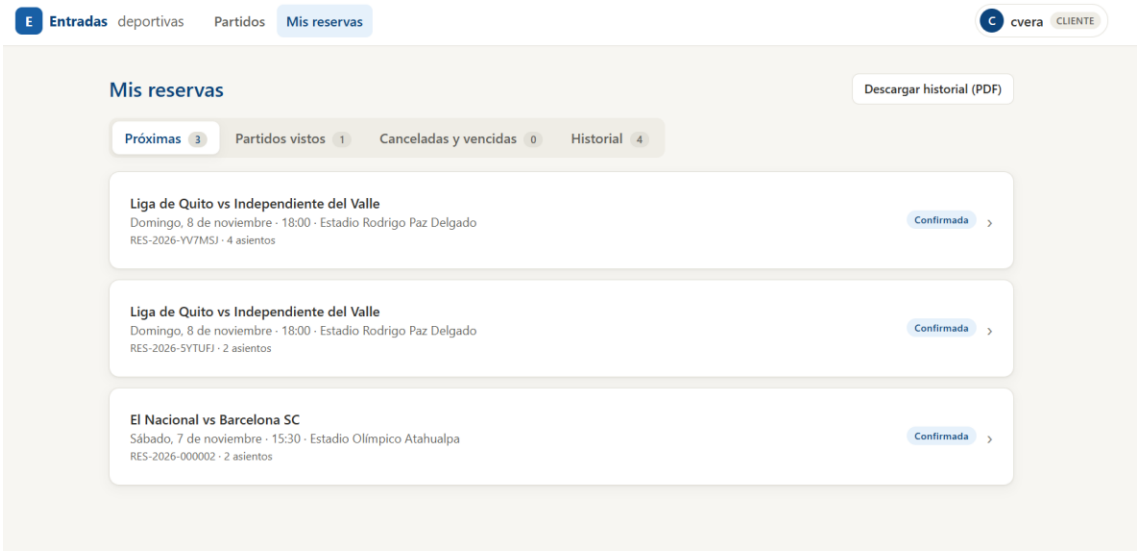
Entradas con QR



Nota. Captura de pantalla del sistema.

Figura 8

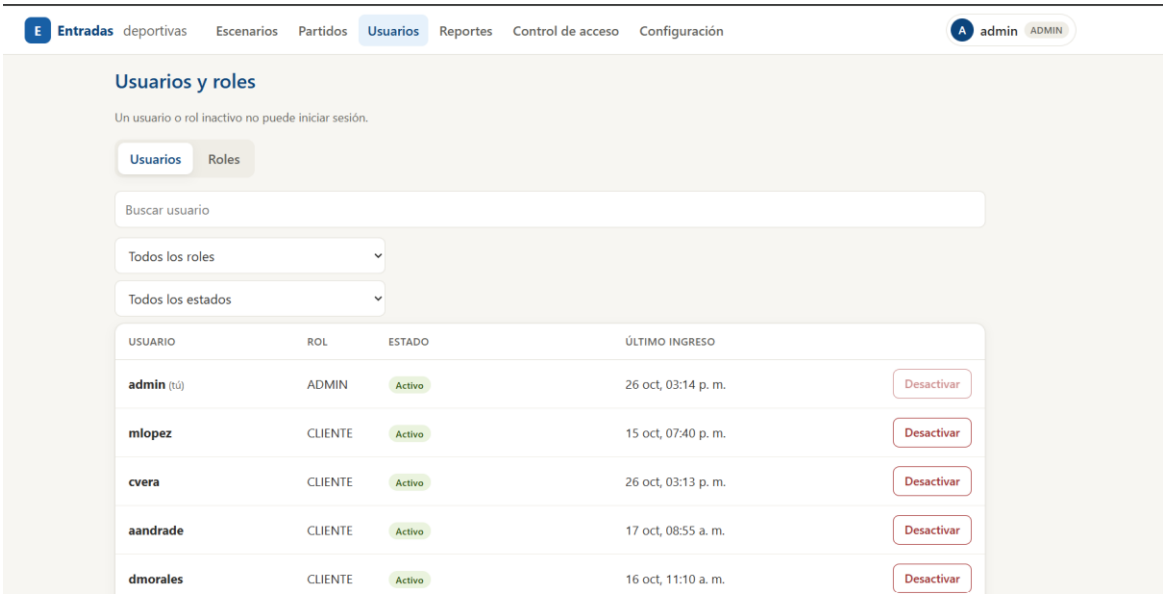
Mis reservas



Nota. Captura de pantalla del sistema.

Figura 9

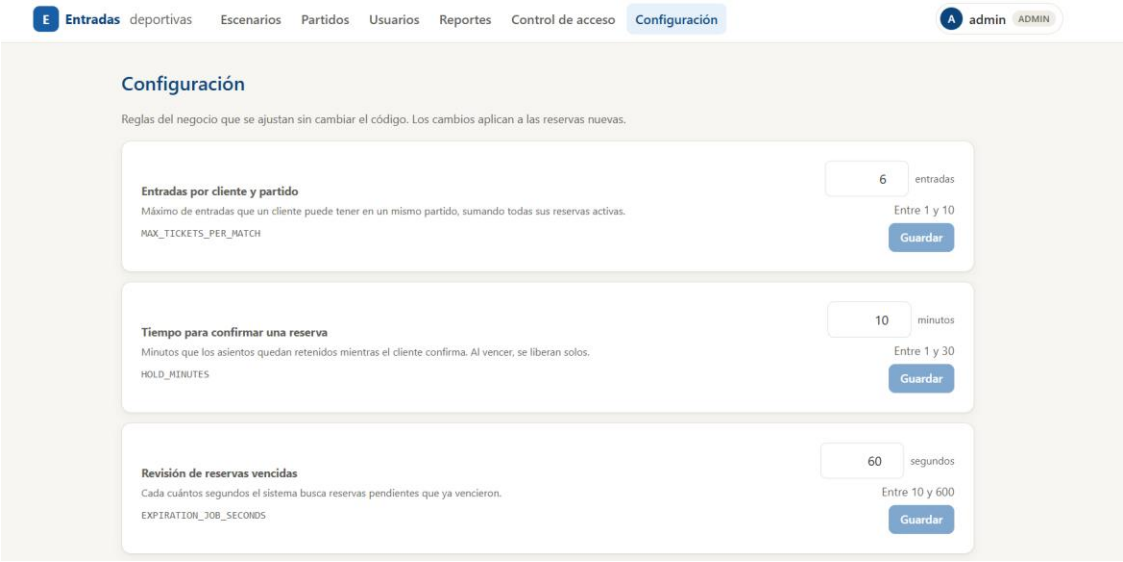
Panel administrativo



Nota. Captura de pantalla del sistema.

Figura 10

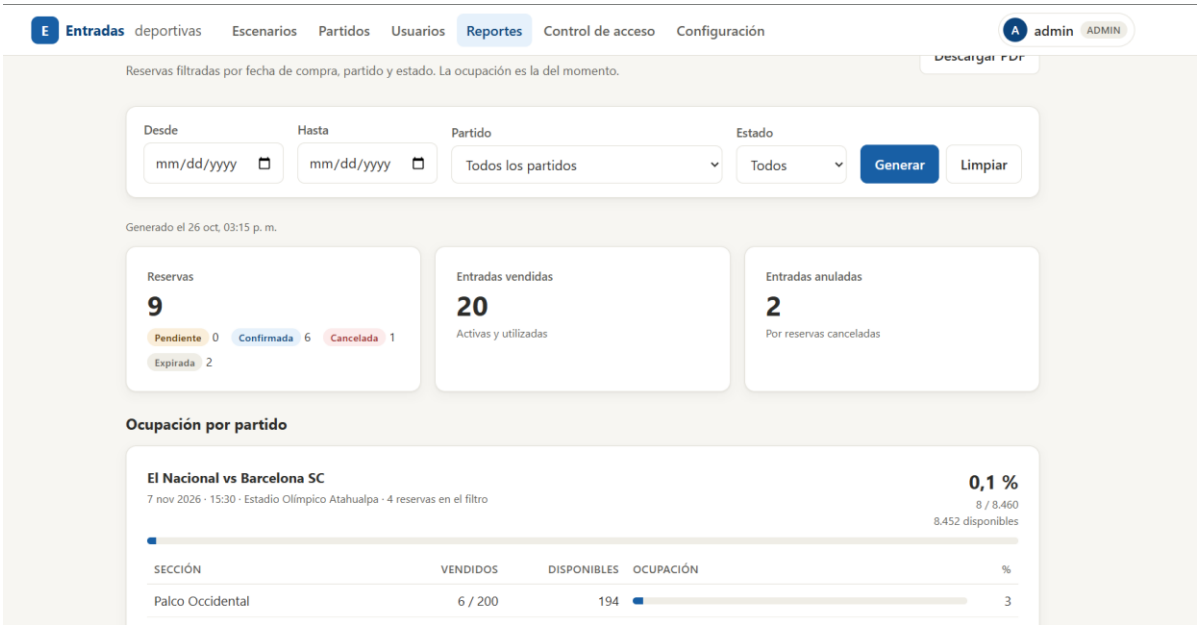
Pantalla de configuración



Nota. Captura de pantalla del sistema.

Figura 11

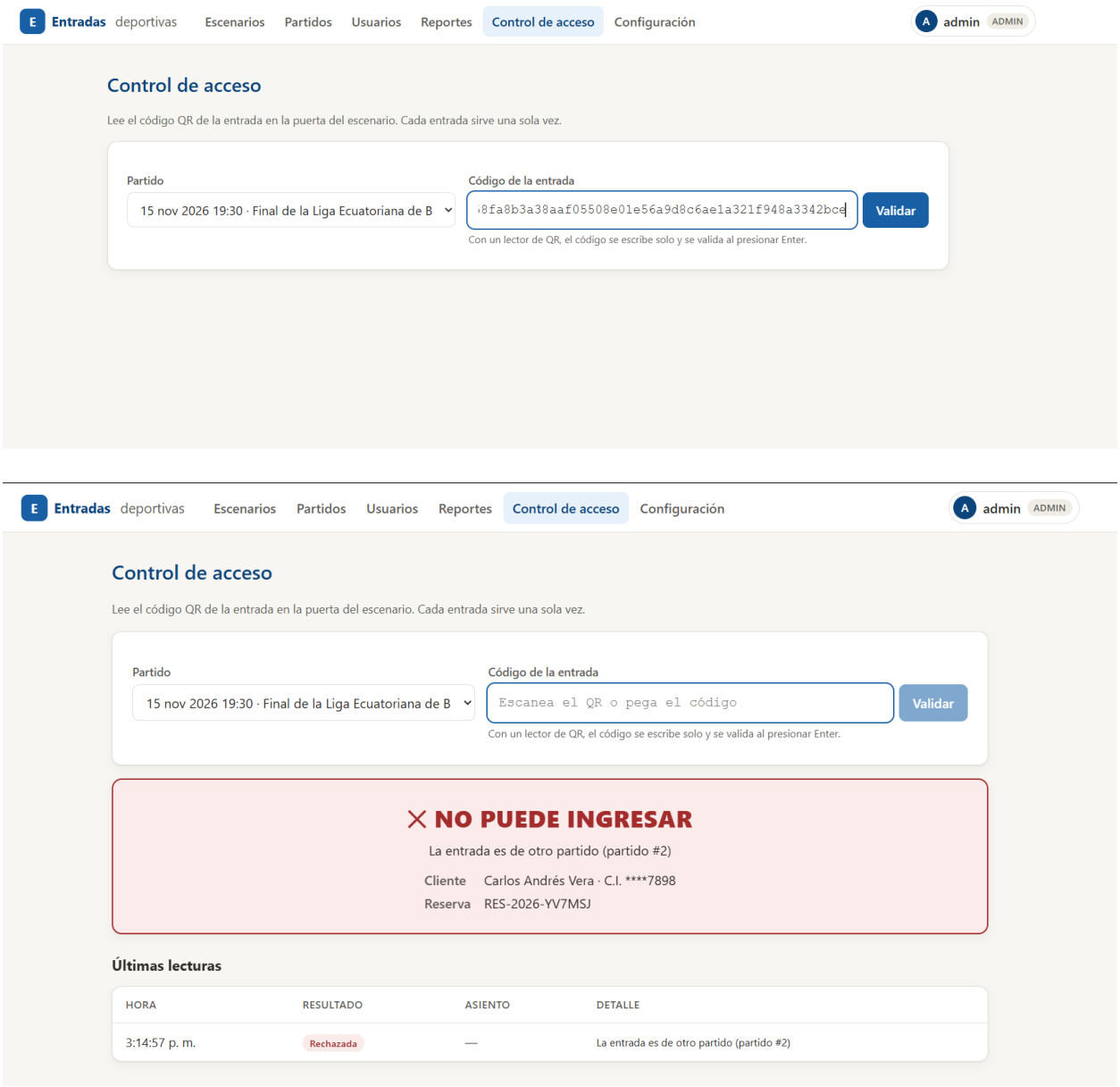
Reportes con indicadores



Nota. Captura de pantalla del sistema.

Figura 12

Control de acceso



Nota. Captura de pantalla del sistema.

Pruebas de base de datos

Tabla 4

Resultado de las pruebas automatizadas de base de datos

Base	Restricciones	Semillas	Total	Resultado
auth_db	26	11	37	37 PASS
match_db	30	16	46	46 PASS
reservation_db	24	13	37	37 PASS
Total	80	40	120	120 PASS · 0 FAIL

Nota. Ejecutadas con database/setup-db.ps1 sobre PostgreSQL 16.

Las 120 pruebas de base de datos se ejecutaron sin fallos (Tabla 4). Las 80 pruebas de restricciones confirmaron que cada regla de integridad rechaza los datos inválidos con la restricción esperada, y las 40 verificaciones de semillas confirmaron que los datos iniciales son coherentes; por ejemplo, que la capacidad de cada sección coincide con los asientos generados y que ningún cliente supera el límite de entradas.

Pruebas funcionales de API

Tabla 5

Escenarios de prueba E2E en Apidog

ID	Escenario	Esperado	Obtenido
E01	Reservar una 7.ª entrada (mlopez, partido 1)	422	422
E02	Reservar un asiento ya vendido (seat 1)	409	409
E03	Cliente llama un endpoint de administrador	403	403
E04	Ver una reserva de otro cliente	404	404
E05	Reserva pendiente vencida al confirmar	409	409
E06	Reservar después del cierre de la venta	422	422

ID	Escenario	Esperado	Obtenido
E07	Login con cuenta bloqueada (pbravo)	423	423

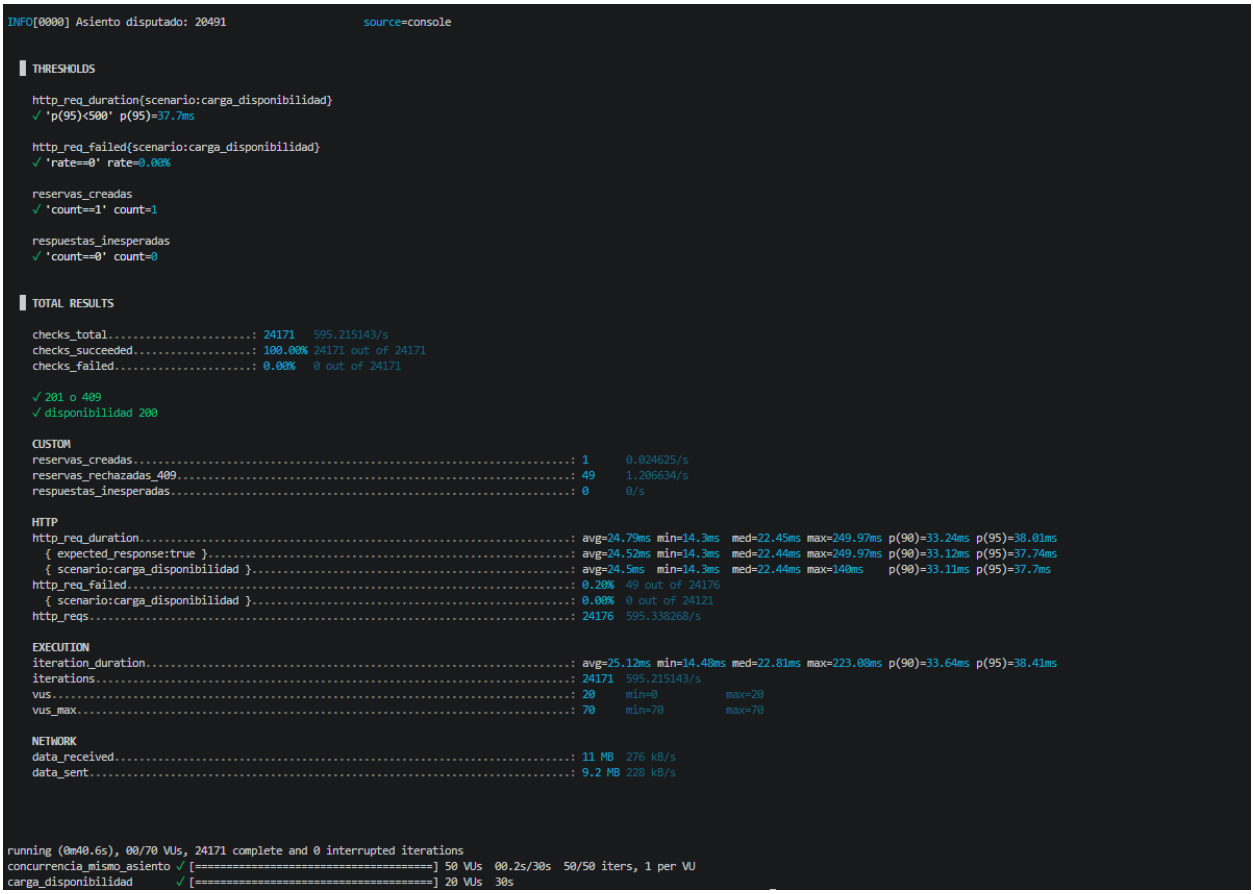
Nota. Elaboración propia. El reporte de ejecución se incluye en el Anexo B.

Se ejecutaron aproximadamente 56 casos de pruebas para verificar el funcionamiento de las APIs, durante las ejecuciones se encontraron que se fueron puliendo a mediada se iba desarrollando el código. Estos errores fueron tanto de negocio como también incumplimiento de contrato. Se contralaron tanto casos positivos como negativos. Se trato de cubrir las reglas de negocio, reservas, cancelaciones, validación de código QR, autenticación y autorización

Prueba de concurrencia y carga

Figura 13

Resumen de la ejecución de la prueba de concurrencia y carga en k6



Nota. Captura de la consola de k6.

En la prueba de concurrencia, 50 usuarios virtuales intentaron reservar el mismo asiento (el asiento 20491 del partido 4) al mismo tiempo. El sistema creó una sola reserva y rechazó las otras 49 con el código 409 Conflict, sin respuestas inesperadas, lo que confirma que la restricción de unicidad UNIQUE(match_id, seat_id) impide la doble venta bajo concurrencia. En la prueba de carga, 20 usuarios virtuales consultaron la disponibilidad del partido de forma continua durante 30 segundos: se atendieron 24 121 solicitudes, aproximadamente 800 por segundo, con un tiempo de respuesta promedio de 24,5 ms, una mediana de 22,4 ms, un percentil 95 de 37,7 ms y un máximo de 140 ms, sin errores. Ambos resultados cumplieron los umbrales

definidos: una única reserva creada, ninguna respuesta inesperada, un percentil 95 inferior a 500 ms y una tasa de error del 0 %. El resumen de la ejecución de k6 se incluye en el Anexo C.

Defectos encontrados durante las pruebas

Tabla 6

Defectos detectados y su corrección

Defecto	Cómo se detectó	Corrección
Escenarios duplicados que solo difieren en una tilde	Prueba exploratoria	Índice único con name_key() y unaccent
Registro con rol CLIENTE inactivo respondía 500	Prueba de roles	Respuesta 422 con mensaje claro
Grilla de 2 080 asientos con scroll horizontal	Prueba de usabilidad	División de secciones en bloques
Partido seguía PROGRAMADO después de su hora	Prueba exploratoria	Cambio de estado automático
Fecha "8 De Noviembre"	Revisión visual	Mayúscula solo en la primera letra
Incumplimiento de contrato en APIs	Pruebas en APIs	Actualización de contratos
El sistema no permitía registrar coliseos	Revisión del caso de estudio	Tipo de escenario ESCENARIO o COLISEO
La venta seguía abierta después de iniciado el partido	Prueba exploratoria	Cierre automático de la venta por hora
El administrador veía “Cliente #4” sin nombre	Prueba exploratoria	Consulta del nombre en auth-service
No había plazo para cancelar una reserva confirmada	Análisis de reglas de negocio	Plazo configurable de 24 h
Un partido cancelado no anulaba sus entradas	Análisis de reglas de negocio	Tarea que cancela reservas y anula entradas
El PDF mostraba el id interno del asiento	Revisión del PDF	Sección, fila y número del asiento

Defecto	Cómo se detectó	Corrección
El menú del administrador se partía en dos líneas	Revisión visual	Ajuste del diseño de la barra

Durante las pruebas se detectaron 13 defectos y vacíos funcionales (Tabla 6). La mayoría no eran fallos de programación, sino reglas de negocio que el caso de estudio no detallaba, como el cierre de la venta o el plazo de cancelación, lo que muestra el valor de probar el sistema desde la perspectiva del negocio y no solo contra los criterios de aceptación. Todos los defectos se corrigieron y se verificaron antes de la entrega.

Discusión de resultados

Los resultados obtenidos permiten concluir que se cumplieron los objetivos planteados para el desarrollo del sistema. Se implementó la capa de datos y se ejecutaron 120 pruebas para comprobar sus restricciones y la consistencia de los datos. También se logró la comunicación entre los tres microservicios mediante API REST, con validación independiente de los tokens JWT. Por otra parte, los módulos de administración y reservas incorporan las reglas de negocio definidas. Finalmente, las pruebas de concurrencia permitieron comprobar que las solicitudes simultáneas no generaran la venta duplicada de un mismo asiento.

Una de las decisiones más importantes durante el desarrollo fue utilizar una base de datos independiente para cada microservicio. Esta organización permite que cada servicio gestione sus propios datos y pueda evolucionar por separado [8]. Sin embargo, también implicó algunos desafíos. Al no compartir una misma base de datos, no es posible establecer llaves foráneas entre servicios ni realizar consultas JOIN directamente entre sus tablas. Por esta razón, las referencias deben comprobarse mediante las API y, cuando se necesita información de varios servicios, es necesario combinar sus respuestas. Esta separación también se relacionó con algunos de los defectos encontrados durante las pruebas, como el problema del «Cliente #4» en el reporte y el identificador del asiento que aparecía en el PDF.

Para la comunicación entre los microservicios se eligió REST, mediante solicitudes síncronas. Esta decisión responde a que, durante una reserva, es necesario consultar la información del partido y comprobar la disponibilidad de los asientos antes de continuar con la operación. No obstante, este enfoque también tiene una limitación: si uno de los servicios necesarios para completar la reserva no responde, la operación no puede continuar. En ese caso, el sistema devuelve una respuesta HTTP 503 en lugar de presentar un error no controlado.

Microsoft [10] señala que la comunicación asíncrona puede reducir la dependencia directa entre microservicios, especialmente cuando una operación no necesita una respuesta inmediata. En este proyecto, por ejemplo, la anuación de las entradas asociadas a un partido cancelado se resolvió mediante una tarea programada, por lo que los cambios no tienen que reflejarse en todos los servicios al mismo instante. Para una versión futura, la incorporación de mensajería permitiría gestionar este tipo de procesos de forma más flexible.

Otro aspecto importante fue el control de las reservas simultáneas. Para reducir el riesgo de que dos usuarios reservaran el mismo asiento, se combinaron una consulta previa de disponibilidad, el bloqueo pesimista y una restricción de unicidad en la base de datos. La consulta permite informar al usuario si el asiento ya está ocupado, mientras que el bloqueo y la restricción protegen la operación cuando se reciben varias solicitudes al mismo tiempo. Se optó por el bloqueo pesimista porque, en situaciones en las que varios usuarios intentan reservar un mismo asiento, resulta importante controlar el acceso al registro durante la operación. En cambio, con un enfoque optimista, las operaciones que entren en conflicto podrían tener que repetirse [16].

La actualización periódica del mapa de asientos también ayuda a mostrar al usuario la disponibilidad más reciente. Sin embargo, esta información puede cambiar mientras el usuario realiza la reserva. Por ello, el mapa no constituye una garantía de que el asiento siga libre al confirmar la operación; esa comprobación debe realizarse en el backend y respaldarse con las restricciones de la base de datos.

En cuanto a la autenticación, se utilizó JWT para que los microservicios pudieran comprobar los tokens sin tener que consultar al servicio de autenticación en cada petición [12]. Esta alternativa reduce las consultas entre servicios, pero presenta una limitación: si una cuenta

se desactiva después de emitir el token, este puede seguir siendo válido hasta su expiración, siempre que no exista otro mecanismo de revocación. En la configuración implementada, la duración máxima del token es de 60 minutos. Además, se utilizó el algoritmo HS256, que requiere compartir la misma clave entre los servicios que verifican los tokens [13]. Para un entorno de producción, sería conveniente evaluar el uso de firma asimétrica, que permite separar la clave utilizada para firmar los tokens de las claves públicas utilizadas para verificarlos.

Finalmente, el sistema presenta algunas limitaciones que pueden abordarse en futuras versiones. Entre ellas se encuentran la ausencia de una pasarela de pagos, la falta de despliegue en un entorno de nube, el uso de una duración fija de 120 minutos para todos los partidos, sin considerar las diferencias entre deportes, y la falta de pruebas unitarias. Estas limitaciones no impiden el funcionamiento de las funcionalidades implementadas, aunque sí delimitan el alcance actual del sistema y permiten identificar aspectos que convendría mejorar antes de utilizarlo en un entorno de producción.

Conclusiones

Se diseñó la capa de datos mediante un modelo entidad-relación distribuido en tres bases de datos independientes, con scripts de creación y datos semillas que representan escenarios con capacidades reales (44 440 asientos en cinco escenarios y un coliseo). Las 120 pruebas de base de datos ejecutadas confirmaron la integridad referencial y las restricciones de negocio, lo que permitió que el sistema iniciara sin datos inconsistentes y que las pruebas funcionales se realizaran sobre información representativa.

Se implementó una arquitectura de tres microservicios (autenticación, partidos y reservas), cada uno cuenta con su propia base de datos y se comunican mediante API REST. Separar las bases de datos obligó a validar las referencias entre servicios por medio de consultas internas protegidas, en lugar de claves foráneas. Esto permitió comprobar el desacoplamiento en la práctica: cuando el servicio de partidos no responde, el historial de reservas y las entradas en PDF siguen disponibles sin los datos del partido. En cambio, si el servicio de reservas no está disponible, los usuarios pueden iniciar sesión y consultar partidos, pero no pueden crear reservas. La autenticación con JWT permitió que cada servicio verificara el rol del usuario de forma independiente.

Se desarrollaron los módulos de administración de usuario, escenarios deportivos y partidos, con soporte para escenarios y coliseos y su distribución en secciones, filas y asientos. El control de acceso basado en roles restringió cada pantalla y cada operación permitida según el perfil establecido: un cliente que intenta ejecutar una operación administrativa recibe una respuesta 403. Además, se aplicó segregación de funciones, ya que el administrador gestiona el sistema, pero no puede reservar entrada, lo que evita conflictos de interés.

Se implementó el proceso de reserva mediante un mapa gráfico de asientos que se actualiza la disponibilidad cada pocos segundos. Se comprobó que esta actualización periódica mejora la experiencia del usuario, pero no basta para impedir la doble reserva, esto se hace mediante una validación en la base de datos de unicidad, por partido y asiento. Las reglas de negocio de límite de entradas por clientes, tiempo de confirmación, cierre de venta y plazo de cancelación se definieron como parámetros de configuración por el administrador. Cada ticket emitido incluye un código QR único, cuya validación se hace en el control de acceso, distinguiendo entradas válidas, ya utilizadas, anuladas o de otro partido.

Se generaron reportes individuales para el cliente, que puede consultar sus reservas por estado, y descargar las entradas en PDF. Para el administrador se crearon reportes generales, con filtros por partidos, fecha y estado. Los indicadores de ocupación y disponibilidad por partido permiten identificar los eventos de mayor demanda y tomar decisiones basadas en datos, como la asignación de recursos o planificación de futuros partidos.

La validación del sistema se realizó por capas. Inicialmente se realizaron pruebas en base de datos para confirmar las restricciones e integridad de los datos. Las pruebas funcionales de API, tomando en cuenta los casos felices y los tristes, verificando el cumplimiento de las reglas de negocio en cada servicio. Se realizaron pruebas de concurrencia intentando reservar el mismo asiento. La interfaz obtuvo una puntuación de 100 en la evaluación de accesibilidad de Lighthouse.

Recomendaciones

A partir de los resultados obtenidos y de las limitaciones identificadas durante el desarrollo, se proponen algunas mejoras para futuras versiones del sistema.

En primer lugar, sería conveniente incorporar un mecanismo de mensajería asíncrona entre los microservicios, utilizando herramientas como RabbitMQ o Kafka. Esto permitiría gestionar eventos como la cancelación de un partido y comunicar el cambio a los servicios relacionados sin depender de una tarea programada. De esta manera, la anulación de las entradas podría procesarse de forma más oportuna y se reduciría la dependencia de la comunicación directa entre los servicios.

En cuanto a la autenticación, se recomienda utilizar tokens de acceso con una duración menor y complementarlos con *refresh tokens* para renovar la sesión cuando corresponda. También sería conveniente evaluar el uso de una firma asimétrica, como RS256, para que únicamente el servicio de autenticación disponga de la clave privada necesaria para emitir los tokens, mientras que los demás servicios puedan verificar su autenticidad mediante la clave pública.

Respecto a las reglas de negocio, se propone permitir la configuración de la duración de los partidos según el deporte, en lugar de aplicar un único valor para todos los eventos. Además, sería útil implementar una clave de idempotencia, como se plantea en la HU-35, para evitar que el reenvío de una misma solicitud genere reservas duplicadas. Otra mejora sería permitir la cancelación de algunas entradas sin necesidad de cancelar toda la reserva, de acuerdo con lo planteado en la HU-36.

Para mejorar la calidad del sistema, se recomienda incorporar pruebas unitarias que comprueben las reglas de negocio de manera aislada y automatizar la ejecución de las pruebas

existentes mediante un flujo de integración continua. Esto facilitaría la detección temprana de errores cuando se realicen cambios en el código y ayudaría a comprobar que las funcionalidades que ya operan correctamente no se vean afectadas.

Finalmente, se recomienda preparar el sistema para su despliegue en un entorno más cercano al de producción. Docker Compose podría utilizarse para organizar los servicios en un entorno controlado, mientras que Kubernetes podría evaluarse si las necesidades de escalabilidad y operación justifican su complejidad adicional. También sería conveniente incorporar monitoreo con Grafana para observar el comportamiento de los servicios, integrar una pasarela de pagos y desarrollar una aplicación móvil para el personal encargado de validar las entradas mediante la cámara del dispositivo.

Referencias bibliográficas

[1] LigaPro, "Liga Profesional de Fútbol del Ecuador," LigaPro. [En línea]. Disponible: https://ligapro.ec/. Accedido: 26 oct. 2026.

[2] V. Altamirano Benítez, F. Paladines Galarza y E. Ruíz, "Comunicación 2.0 en el fútbol profesional del Ecuador," *Revista Ibérica de Sistemas e Tecnologias de Informação*, no. E40. 2021.

[3] M. A. Cayetano Quinde y M. A. Flores Villamarín, "Diseño de un sistema para la adquisición de entradas a los escenarios de fútbol del país," tesis de pregrado, Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador, 2017. [En línea]. Disponible: https://dspace.espol.edu.ec/bitstream/123456789/44440/1/T-106410.pdf

[4] Carrera de Ingeniería en Sistemas de Información, Universidad Central del Ecuador, "Sistema de gestión de entradas para eventos deportivos," examen complexivo, componente práctico, Quito, Ecuador, 2026.

[5] K. T. Gómez Suárez, R. Anaya y A. F. Cano, "Un acercamiento a los microservicios," *UNACIENCIA. Revista de Estudios e Investigaciones*, vol. 10, no. 19, pp. 116–126, 2018. [En línea]. Disponible: https://revistas.unac.edu.co/ojs/index.php/unaciencia/article/download/235/214

[6] D. A. Barrios Contreras, "Arquitectura de microservicios," *Tecnología, Investigación y Academia*, vol. 6, no. 1, pp. 37–46, 2018. [En línea]. Disponible: https://revistas.udistrital.edu.co/index.php/tia/article/download/9687/pdf

[7] J. Lewis y M. Fowler, "Microservices: A definition of this new architectural term," martinfowler.com, 25 mar. 2014. [En línea]. Disponible: https://martinfowler.com/articles/microservices.html. Accedido: 26 oct. 2026.

[8] C. Richardson, “Pattern: Database per service,” microservices.io. [En línea]. Disponible: https://microservices.io/patterns/data/database-per-service.html. Accedido: 26 oct. 2026.

[9] C. Richardson, *Microservices Patterns*. Shelter Island, NY, EE. UU.: Manning, 2018.

[10] Microsoft, “Comunicación en una arquitectura de microservicios,” Microsoft Learn, 13 abr. 2022. [En línea]. Disponible: https://learn.microsoft.com/es-es/dotnet/architecture/microservices/architect-microservice-container-applications/communication-in-microservice-architecture. Accedido: 26 oct. 2026.

[11] OWASP Foundation, “Authorization cheat sheet,” OWASP Cheat Sheet Series. [En línea]. Disponible: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html. Accedido: 26 oct. 2026.

[12] M. Jones, J. Bradley y N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, may. 2015. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc7519

[13] M. Jones, “JSON Web Algorithms (JWA),” Internet Engineering Task Force, RFC 7518, may. 2015. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc7518

[14] OWASP Foundation, “Password storage cheat sheet,” OWASP Cheat Sheet Series. [En línea]. Disponible: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Accedido: 26 oct. 2026.

[15] N. Provos y D. Mazières, “A future-adaptable password scheme,” en *Proc. 1999 USENIX Annu. Tech. Conf.*, Monterey, CA, EE. UU., 1999, pp. 81–91.

[16] A. Silberschatz, H. F. Korth y S. Sudarshan, *Database System Concepts*, 7.ª ed. Nueva York, NY, EE. UU.: McGraw-Hill, 2019.

[17] R. T. Fielding, "Architectural styles and the design of network-based software architectures," tesis doctoral, University of California, Irvine, CA, EE. UU., 2000. [En línea]. Disponible: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[18] OpenAPI Initiative, "OpenAPI Specification v3.0.3." [En línea]. Disponible: https://spec.openapis.org/oas/v3.0.3. Accedido: 26 oct. 2026.

[19] M. Nottingham, E. Wilde y S. Dalal, "Problem details for HTTP APIs," Internet Engineering Task Force, RFC 9457, jul. 2023. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc9457

[20] Meta Open Source, "React." [En línea]. Disponible: https://react.dev/. Accedido: 26 oct. 2026.

[21] Mozilla Developer Network, "Cross-Origin Resource Sharing (CORS)." [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS. Accedido: 26 oct. 2026.

[22] Vite, "Server options: server.proxy." [En línea]. Disponible: https://vite.dev/config/server-options. Accedido: 26 oct. 2026.

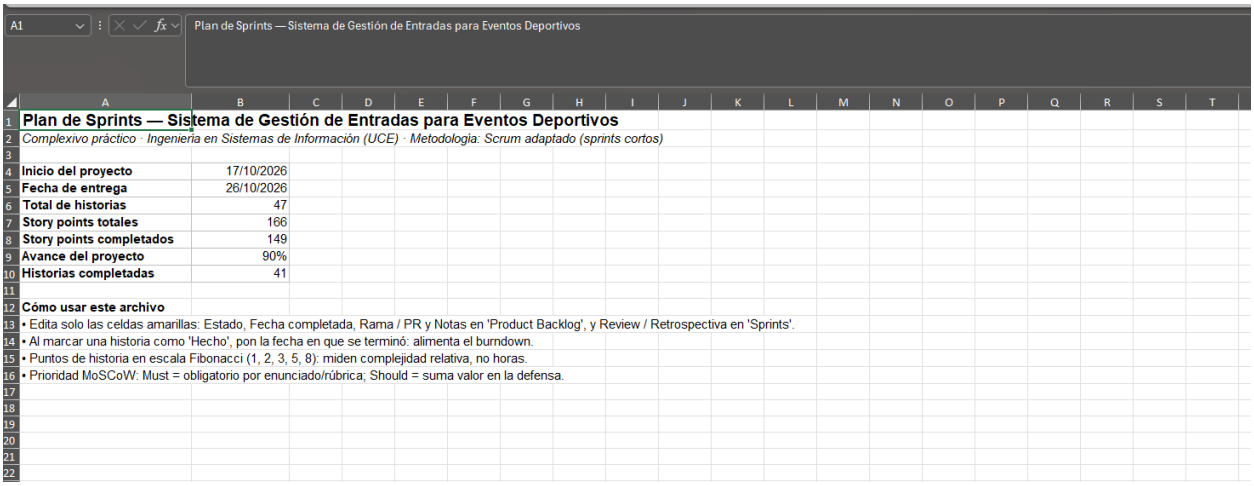
[23] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9.ª ed. Nueva York, NY, EE. UU.: McGraw-Hill, 2020.

[24] K. Schwaber y J. Sutherland, "La Guía Scrum: La guía definitiva de Scrum: Las reglas del juego," nov. 2020. [En línea]. Disponible: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish.pdf

Anexos

Anexo A

Plan de sprints del proyecto



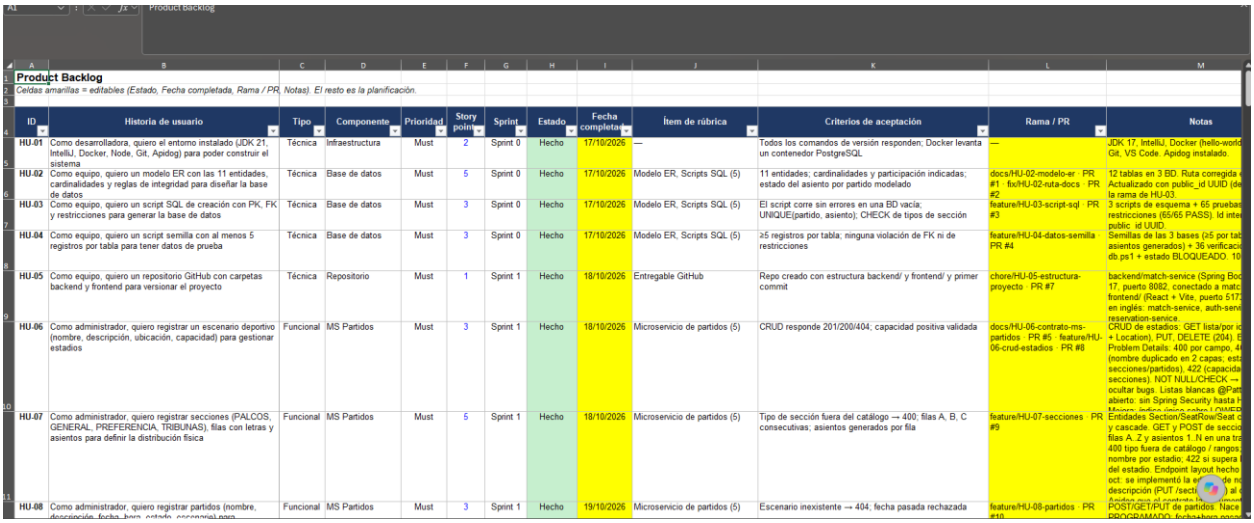
Plan de Sprints — Sistema de Gestión de Entradas para Eventos Deportivos

Complexivo práctico · Ingeniería en Sistemas de Información (UCE) · Metodología: Scrum adaptado (sprints cortos)

Inicio del proyecto	17/10/2026
Fecha de entrega	26/10/2026
Total de historias	47
Story points totales	166
Story points completados	149
Avance del proyecto	90%
Historias completadas	41

Cómo usar este archivo

- Edita solo las celdas amarillas: Estado, Fecha completada, Rama / PR y Notas en 'Product Backlog', y Review / Retrospectiva en 'Sprints'.
- Al marcar una historia como 'Hecho', pon la fecha en que se terminó: alimenta el burndown.
- Puntos de historia en escala Fibonacci (1, 2, 3, 5, 8): miden complejidad relativa, no horas.
- Prioridad MoSCoW: Must = obligatorio por enunciado/rúbrica; Should = suma valor en la defensa.



Product Backlog

Celdas amarillas = editables (Estado, Fecha completada, Rama / PR, Notas). El resto es la planificación.

ID	Historia de usuario	Tipo	Componente	Prioridad	Story points	Sprint	Estado	Fecha completada	Ítem de rúbrica	Criterios de aceptación	Rama / PR	Notas
HU-01	Como desarrolladora, quiero el entorno instalado (JDK 21, IntelliJ, Docker, Node, Git, Apidog) para poder construir el sistema	Técnica	Infraestructura	Must	2	Sprint 0	Hecho	17/10/2026	—	Todos los comandos de versión responden; Docker levanta un contenedor PostgreSQL	—	JDK 17, IntelliJ, Docker (hello-world), Git, VS Code. Apidog instalado.
HU-02	Como equipo, quiero un modelo ER con las 11 entidades, cardinalidades y reglas de integridad para diseñar la base de datos	Técnica	Base de datos	Must	5	Sprint 0	Hecho	17/10/2026	Modelo ER, Scripts SQL (5)	11 entidades; cardinalidades y participación indicadas; estado del asiento por partido modelado	docs/HU-02-modelo-er · PR #1 · fix/HU-02-ruta-docs · PR #2	12 tablas en 3 BD. Ruta corregida d Actualizado con public_id UUID (de la rama de HU-03.
HU-03	Como equipo, quiero un script SQL de creación con PK, FK y restricciones para generar la base de datos	Técnica	Base de datos	Must	3	Sprint 0	Hecho	17/10/2026	Modelo ER, Scripts SQL (5)	El script corre sin errores en una BD vacía; UNIQUE(partido, asiento); CHECK de tipos de sección	feature/HU-03-script-sql · PR #3	3 scripts de esquema + 65 pruebas restricciones (65/65 PASS). Id inte public_id UUID.
HU-04	Como equipo, quiero un script semilla con al menos 5 registros por tabla para tener datos de prueba	Técnica	Base de datos	Must	3	Sprint 0	Hecho	17/10/2026	Modelo ER, Scripts SQL (5)	≥5 registros por tabla; ninguna violación de FK ni de restricciones	feature/HU-04-datos-semilla · PR #4	Semillas de las 3 bases (≥5 por tab asientos generados) + 36 verificaci db.ps1 + estado BLOQUEADO. 10
HU-05	Como equipo, quiero un repositorio GitHub con carpetas backend y frontend para versionar el proyecto	Técnica	Repositorio	Must	1	Sprint 1	Hecho	18/10/2026	Entregable GitHub	Repo creado con estructura backend/ y frontend/ y primer commit	chore/HU-05-estructura-proyecto · PR #7	backend/match-service (Spring Boo 17, puerto 8082, conectado a matc frontend/ (React + Vite, puerto 517 en inglés: match-service, auth-serv reservation-service.
HU-06	Como administrador, quiero registrar un escenario deportivo (nombre, descripción, ubicación, capacidad) para gestionar estadios	Funcional	MS Partidos	Must	3	Sprint 1	Hecho	18/10/2026	Microservicio de partidos (5)	CRUD responde 201/200/404; capacidad positiva validada	docs/HU-06-contrato-ms-partidos · PR #5 · feature/HU-06-crud-estadios · PR #6	CRUD de estadios: GET lista/por id + Location), PUT, DELETE (204). E Problem Details: 400 por campo, 4 (nombre duplicado en 2 capas; est secciones/partidos), 422 (capacida secciones). NOT NULL/CHECK → ocultar bugs. Listas blancas @Par abierto: sin Spring Security hasta H
HU-07	Como administrador, quiero registrar secciones (PALCOS, GENERAL, PREFERENCIA, TRIBUNAS), filas con letras y asientos para definir la distribución física	Funcional	MS Partidos	Must	5	Sprint 1	Hecho	18/10/2026	Microservicio de partidos (5)	Tipo de sección fuera del catálogo → 400; filas A, B, C consecutivas; asientos generados por fila	feature/HU-07-secciones · PR #9	Entidades Section/SeatRow/Seat c y cascade. GET y POST de secci filas A..Z y asientos 1..N en una tra 400 tipo fuera de catálogo / rangos nombre por estadio; 422 si supera del estadio. Endpoint layout hecho oct. se implementó la e descripción (PUT /sect
HU-08	Como administrador, quiero registrar partidos (nombre,	Funcional	MS Partidos	Must	3	Sprint 1	Hecho	19/10/2026	Microservicio de partidos (5)	Escenario inexistente → 404; fecha pasada rechazada	feature/HU-08-partidos · PR #10	POST/GET/PUT de partidos. Nace

Sprints

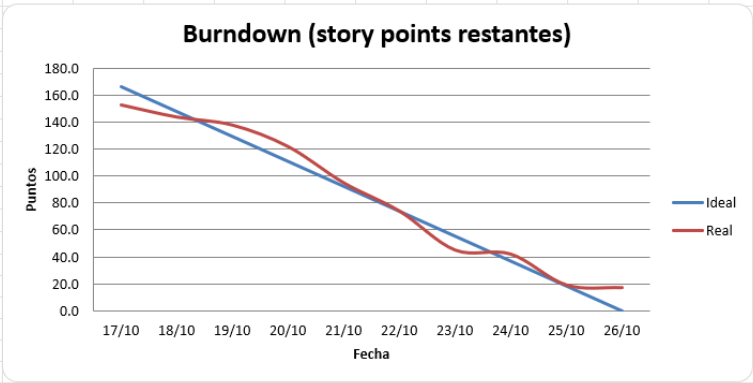
Los puntos y el avance se calculan desde el Product Backlog. Review y Retrospectiva se llenan al cerrar cada sprint.

Sprint	Objetivo (Sprint Goal)	Inicio	Fin	Incremento entregable	Historias	Puntos planificados	Puntos completados	Avance	Sprint Review	Retrospectiva
Sprint 0	Base de datos y entorno	17/10/2026	17/10/2026	Entorno instalado, modelo ER, script SQL de creación y semillas	4	13	13	100%	Incremento: modelo ER (12 tablas, 3 BD), scripts SQL con restricciones nombradas, datos semilla (≥5 por tabla) y setup-db.ps1. 106/106 pruebas PASS. La base ya soporta autenticación y autorización; la auditoría queda pendiente para el Sprint 2. HU-05 pasa al Sprint 1 (faltan carpetas backend/ y frontend/). Resumen completo en	Salió bien: crear las bases de datos y encontrar las relaciones entre las tablas. Costó: recordar el modelo entidad-relación y saber dónde va la FK. Acción de mejora: probar en Apidog cada endpoint apenas esté listo, no al final del sprint.
Sprint 1	Microservicios Partidos y Usuarios/Auth	18/10/2026	20/10/2026	Partidos y Auth funcionando con Swagger, JWT y roles	10	31	31	100%	Incremento: match-service (estadios, secciones con filas y asientos, partidos con máquina de estados, catálogo con filtros) y auth-service (registro con credenciales por correo, login JWT con bloqueo, administración de usuarios y roles), ambos con Swagger. 31/31 puntos. 8 bugs encontrados y corregidos con pruebas (valores límite, transición de estados, seguridad). Riesgo abierto: match-service aún no valida el JWT. Resumen completo en	Salió bien: se completaron las 10 historias y los dos microservicios funcionan. Costó: surgieron dudas con conceptos nuevos (Java, JPA, JWT) y con Git (rama equivocada). Acción de mejora: hacer las cosas más rápido: revisar la rama antes de empezar, cerrar cada HU con su PR y probar cada endpoint apenas esté listo.
Sprint 2	Microservicio Reservas	20/10/2026	22/10/2026	Reservas con regla de 6, expiración, QR y reportes PDF	10	43	43	100%	Incremento: reservation-service completo (disponibilidad, reservar con máximo 6, expiración automática, confirmar con tickets QR, cancelar, mis reservas, entradas y reporte del administrador en JSON y PDF), integrado con match-service	Salió bien: se terminaron todas las historias comprometidas; fue interesante, en especial entender los PDF. Costó: llevó más tiempo del planificado (un día más y nuevas historias descubiertas al probar).

Burndown del proyecto

Real = puntos totales − puntos de historias 'Hecho' con fecha completada hasta ese día.

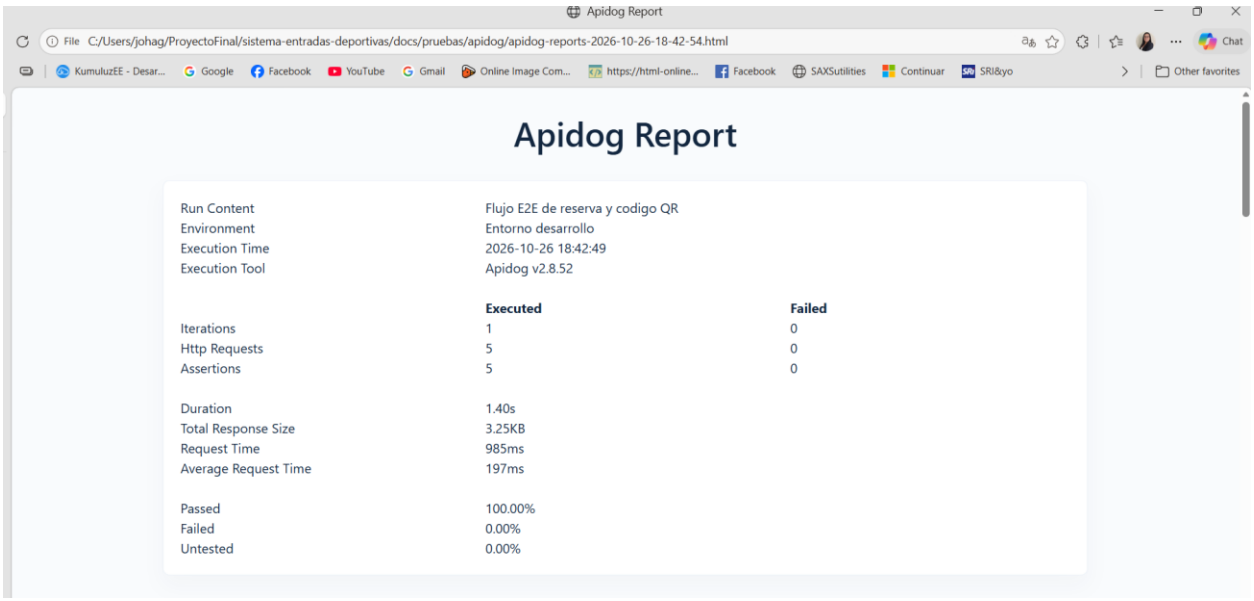
Fecha	Ideal	Real
17/10	166.0	153
18/10	147.6	144
19/10	129.1	138
20/10	110.7	122
21/10	92.2	95
22/10	73.8	74
23/10	55.3	45
24/10	36.9	42
25/10	18.4	19
26/10	0.0	17

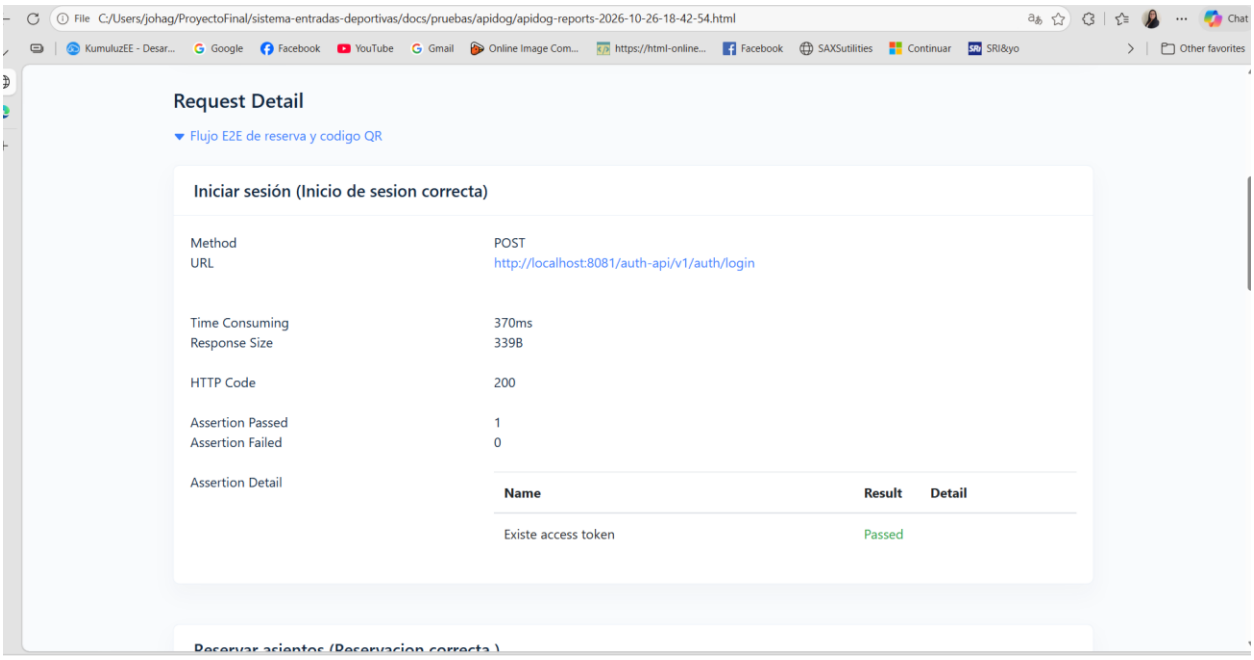


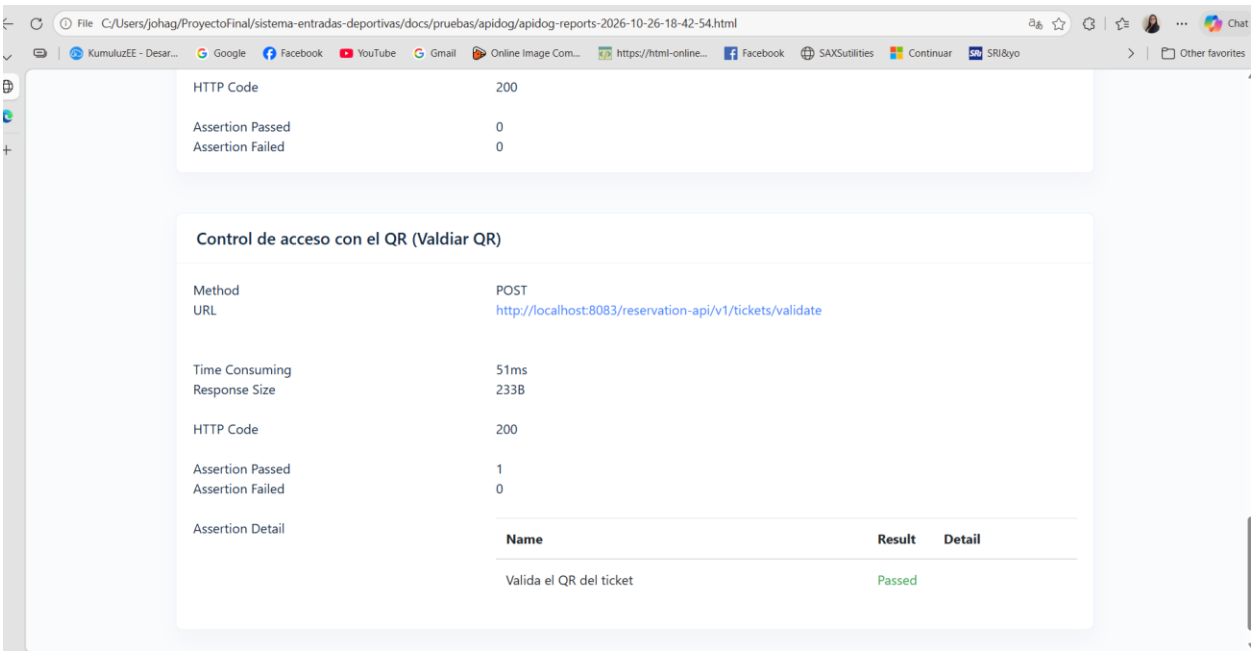
El archivo completo está en docs/gestion del repositorio.

Anexo B

Reporte de ejecución de pruebas de API



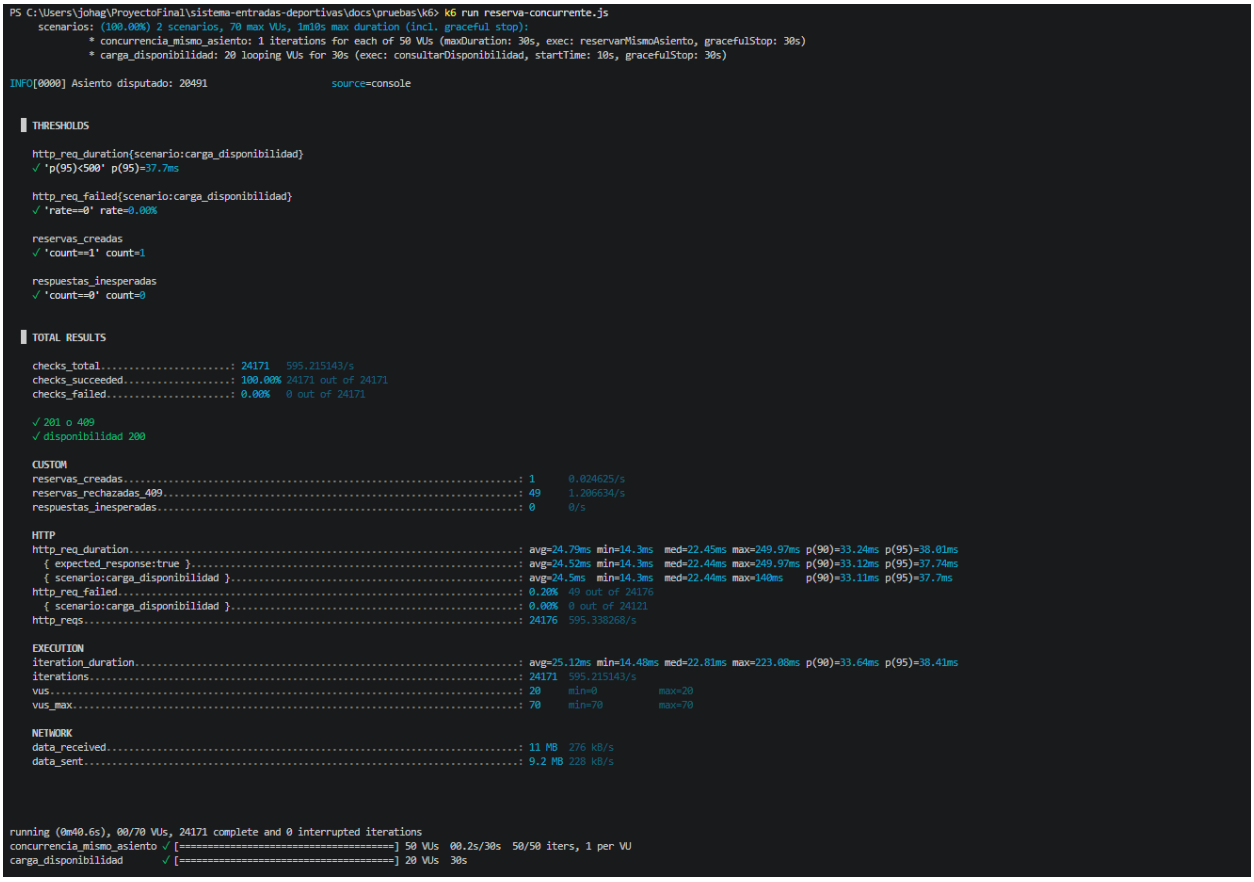




El archivo completo está en docs/pruebas/apidog del repositorio

Anexo C

Resumen de la prueba de concurrencia y carga



```
PS C:\Users\johag\ProyectoFinal\sistema-entradas-deportivas\docs\pruebas\k6> k6 run reserva-concurrente.js
     scenarios: (100.00%) 2 scenarios, 70 max VUs, 1m10s max duration (incl. graceful stop):
              * concurrencia_mismo_asiento: 1 iterations for each of 50 VUs (maxDuration: 30s, exec: reservarMismoAsiento, gracefulStop: 30s)
              * carga_disponibilidad: 20 looping VUs for 30s (exec: consultarDisponibilidad, startTime: 10s, gracefulStop: 30s)

INFO[0000] Asiento disputado: 20491                      source=console


  THRESHOLDS

    http_req_duration{scenario:carga_disponibilidad}
    ✓ 'p(95)<500' p(95)=37.7ms

    http_req_failed{scenario:carga_disponibilidad}
    ✓ 'rate==0' rate=0.00%

    reservas_creadas
    ✓ 'count==1' count=1

    respuestas_inesperadas
    ✓ 'count==0' count=0


  TOTAL RESULTS

    checks_total.....................: 24171   595.215143/s
    checks_succeeded.................: 100.00% 24171 out of 24171
    checks_failed....................: 0.00%   0 out of 24171

    ✓ 201 o 409
    ✓ disponibilidad 200

    CUSTOM
    reservas_creadas.................................................: 1     0.024625/s
    reservas_rechazadas_409..........................................: 49    1.206634/s
    respuestas_inesperadas...........................................: 0     0/s

    HTTP
    http_req_duration................................................: avg=24.79ms min=14.3ms  med=22.45ms max=249.97ms p(90)=33.24ms p(95)=38.01ms
      { expected_response:true }.....................................: avg=24.52ms min=14.3ms  med=22.44ms max=249.97ms p(90)=33.12ms p(95)=37.74ms
      { scenario:carga_disponibilidad }..............................: avg=24.5ms  min=14.3ms  med=22.44ms max=140ms    p(90)=33.11ms p(95)=37.7ms
    http_req_failed..................................................: 0.20%  49 out of 24176
      { scenario:carga_disponibilidad }..............................: 0.00%  0 out of 24121
    http_reqs........................................................: 24176  595.338268/s

    EXECUTION
    iteration_duration...............................................: avg=25.12ms min=14.48ms med=22.81ms max=223.08ms p(90)=33.64ms p(95)=38.41ms
    iterations.......................................................: 24171  595.215143/s
    vus..............................................................: 20     min=0           max=20
    vus_max..........................................................: 70     min=70          max=70

    NETWORK
    data_received....................................................: 11 MB  276 kB/s
    data_sent........................................................: 9.2 MB 228 kB/s

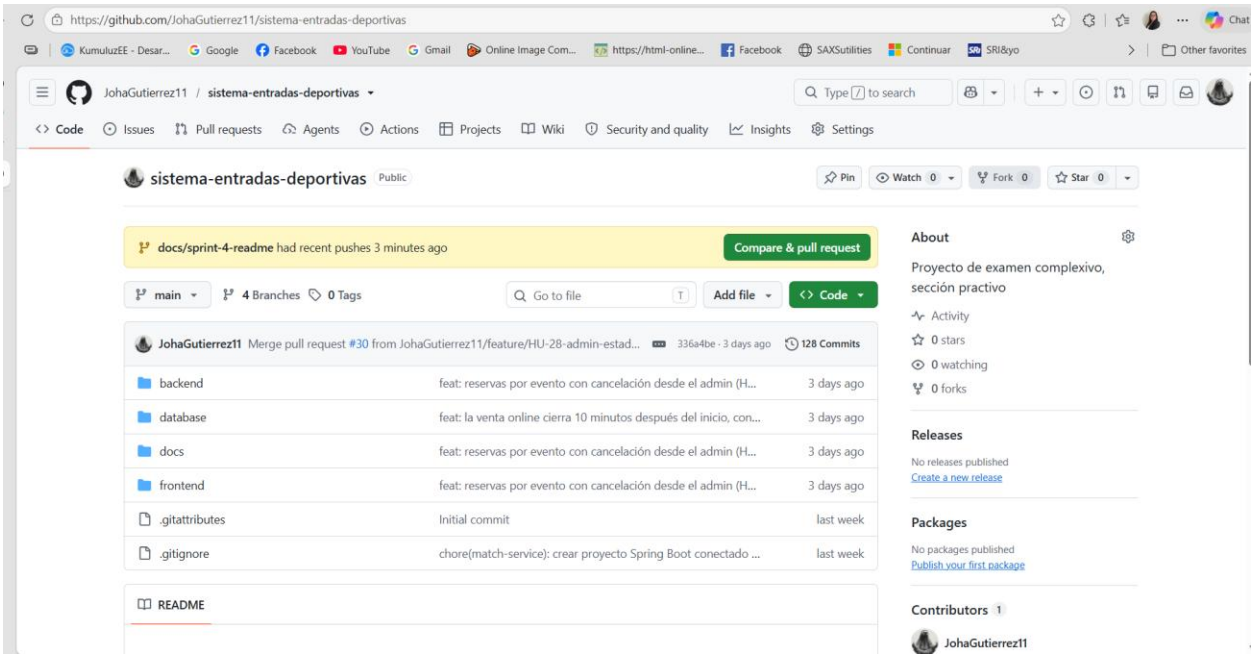


running (0m40.6s), 00/70 VUs, 24171 complete and 0 interrupted iterations
concurrencia_mismo_asiento ✓ [======================================] 50 VUs  00.2s/30s  50/50 iters, 1 per VU
carga_disponibilidad       ✓ [======================================] 20 VUs  30s
```

Para ver el script de pruebas ver el archivo completo en docs/pruebas/k6 del repositorio

Anexo D

Repositorio del proyecto



Enlace del repositorio: [JohaGutierrez11/sistema-entradas-deportivas: Proyecto de examen complexivo, parte practica con caso de estudio.]

En el repositorio se encuentra el backend, frontend, database y documentación.

Anexo E

Contratos de la API: endpoints principales

Los contratos completos, en formato OpenAPI 3, están en la carpeta docs/apis/ del repositorio. La Tabla E1 resume los endpoints principales de cada microservicio y el rol que puede usarlos.

Tabla E1

Endpoints principales por microservicio

Servicio	Método	Ruta	Acceso
auth-service	POST	/auth-api/v1/auth/register	Público
	POST	/auth-api/v1/auth/login	Público
	POST	/auth-api/v1/auth/change-password	Autenticado
	GET	/auth-api/v1/users	Administrador
	PATCH	/auth-api/v1/users/{userId}/status	Administrador
	GET	/auth-api/v1/clients?ids=	Administrador
	GET · PATCH	/auth-api/v1/roles · /roles/{roleId}/status	Administrador
match-service	GET	/match-api/v1/stadiums · /stadiums/{id}	Público
	POST · PUT · DELETE	/match-api/v1/stadiums · /stadiums/{id}	Administrador
	GET	/match-api/v1/stadiums/{id}/layout	Público
	GET · POST	/match-api/v1/stadiums/{id}/sections	Administrador
	PUT	/match-api/v1/sections/{id} · /sections/{id}/box	Administrador
	GET	/match-api/v1/matches · /matches/{id}	Público
	POST · PUT	/match-api/v1/matches · /matches/{id}	Administrador
	POST	/match-api/v1/internal/seats/validate · /seats/labels	Interno (X-Internal-Key)
reservation-service	GET	/reservation-api/v1/matches/{id}/availability	Público
	POST	/reservation-api/v1/reservations	Cliente
	POST	/reservation-api/v1/reservations/{publicId}/confirm	Cliente
	POST	/reservation-api/v1/reservations/{publicId}/cancel	Cliente o administrador
	GET	/reservation-api/v1/reservations/me · /me/pdf	Cliente
	GET	/reservation-api/v1/reservations/{publicId}/tickets/pdf	Cliente (propia) o administrador
	GET	/reservation-api/v1/tickets/{publicId}/qr	Cliente (propio) o administrador

Servicio	Método	Ruta	Acceso
	GET	/reservation-api/v1/reports/reservations · /reports/reservations/pdf	Administrador
	GET	/reservation-api/v1/reports/matches/{id}/reservations	Administrador
	GET · PUT	/reservation-api/v1/parameters · /parameters/{code}	Administrador
	POST	/reservation-api/v1/tickets/validate	Administrador

Anexo F

Manual de instalación

Requisitos: Docker Desktop, JDK 17 y Node.js 22 LTS.

1. Base de datos. Desde la carpeta database, ejecutar .\setup-db.ps1. El script crea el contenedor de PostgreSQL 16 (puerto 5433), las tres bases de datos, sus tablas y los datos semilla, y ejecuta las 120 pruebas de base de datos. También sirve para volver al estado inicial después de una demostración.
2. Correo de desarrollo. Ejecutar docker run -d --name mailpit -p 2525:1025 -p 8025:8025 axllent/mailpit. Las contraseñas temporales del registro llegan a la bandeja http://localhost:8025.
3. Microservicios. En tres terminales y en este orden, desde la carpeta de cada servicio, ejecutar .\mvnw spring-boot:run: backend\auth-service (8081), backend\match-service (8082) y backend\reservation-service (8083).
4. Frontend. Desde la carpeta frontend, ejecutar npm install y luego npm run dev. La aplicación queda disponible en http://localhost:5173.

Variables de entorno. Los servicios arrancan con valores de desarrollo. En otro ambiente deben definirse JWT_SECRET (en los tres servicios, con al menos 32 caracteres), INTERNAL_API_KEY (en match-service y reservation-service) y MATCH_SERVICE_URL (en reservation-service).

Usuarios de prueba (solo desarrollo): admin / Admin2026* para el administrador, y mlopez, cvera y aandrade / Cliente2026* para los clientes.